

Learn Google Earth Engine

全套教学课程

面向遥感、GIS、生态与林业科研人员的 JavaScript Code Editor 实战手册

2026.07

目录

1	GEE 基础介绍与平台准入.....	4
1.1	GEE 是什么、为什么值得学	4
1.2	云端计算原理	6
1.3	账号注册、授权与进入 Code Editor	8
1.4	注册失败、权限与账号问题排查.....	11
1.5	学习路线与核心数据类型概览.....	12
2	GEE Code Editor 教学	14
2.1	print、点线面矩形与 Map.addLayer.....	14
2.2	五类核心数据结构.....	15
2.3	筛选、去云、镶嵌与裁剪	17
2.4	NDVI、EVI、NDBI、NDWI 与 SAVI.....	19
2.5	均值、最大值、最小值与月度/年度合成.....	20
2.6	图层配色、可视化与 Inspector 判读.....	23
2.7	ui.Chart 图表绘制	24
2.8	导出影像、矢量和表格	25
2.9	客户端/服务器、map 与性能调试.....	26
2.10	综合练习.....	27
3	从个人数据到公开应用与 AI 辅助编程.....	29
3.1	上传、检查与管理个人 Asset	29
3.2	把个人 Asset 用作 ROI 与分区统计	32
3.3	建立可解释、可比较的可视化.....	36
3.4	共享脚本、GitHub、Assets 与公开 Earth Engine App	38
3.5	六遍读懂一段 GEE 代码.....	41
3.6	GPT × GEE 的 Vibe Coding：从提示词到科学验收	43
4	完整科研实战：2018—2026 森林植被动态监测	48
4.1	项目设计、研究问题与数据选择.....	48
4.2	研究区、分区与森林样本框	49
4.3	Sentinel-2/Landsat 批量获取与预处理	50
4.4	年度最大 NDVI 合成.....	50

4.5	线性趋势、Sen' s slope 与 Kendall tau	51
4.6	变化提取与各区域植被占比	52
4.7	完整项目代码与复现实验	52
附录	官方资料与数据引用	62

1 GEE 基础介绍与平台准入

1.1 GEE 是什么、为什么值得学

1.1.1 GEE 介绍

Google Earth Engine (GEE) 是面向地球科学与地理空间分析的云平台，其对应的网址是：<https://code.earthengine.google.com>。它把大规模公开遥感数据目录、并行计算能力、地理空间算法和在线开发环境放在一起。用户通常不是先下载几百景影像再在本地逐景处理，而是向服务器描述“需要哪些数据、执行什么计算、输出什么结果”。

GEE 的主要优势包括：

1. 数据集中：Landsat、Sentinel、MODIS、气候、地形、土地覆盖等数据可直接调用。
2. 计算靠近数据：筛选、合成和统计在云端完成，降低本地存储和预处理负担。
3. 可重复：代码保存了数据集 ID、时间、空间范围、算法与参数。
4. 易扩展：从一个样地改到一个省，通常只需替换研究区并评估计算规模。
5. 易分享：脚本、App、导出表和影像可以形成完整复现链。

典型应用场景包括森林变化、植被物候、火烧迹地、干旱、农作物、水体、城市扩张、生态恢复、气候暴露、地表温度和灾害监测。GEE 擅长大范围、多时相、规则明确的栅格运算；它不是桌面 GIS 的完全替代品。复杂制图排版、精细人工编辑、离线保密环境和超定制本地模型仍可能需要 QGIS、ArcGIS、R 或 Python。

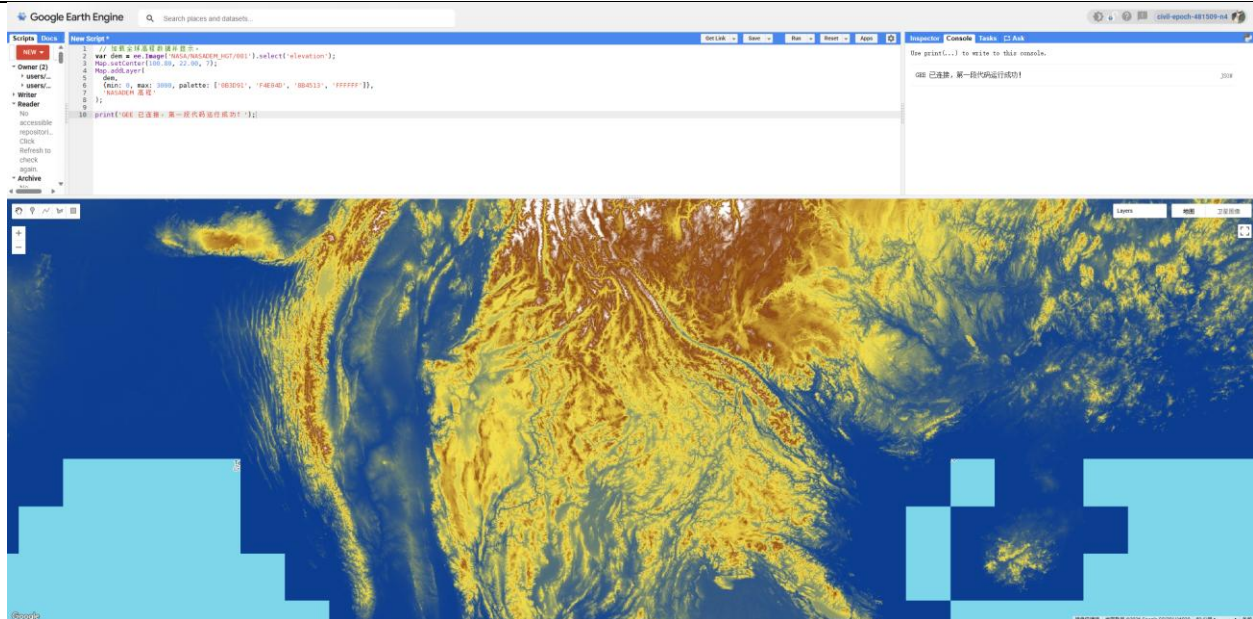


1.1.2 应用示例

可运行代码如下：

```
// 加载全球高程数据并显示。

var dem = ee.Image('NASA/NASADEM_HGT/001').select('elevation');
Map.setCenter(100.80, 22.00, 7);
Map.addLayer(
  dem,
  {min: 0, max: 3000, palette: ['0B3D91', 'F4E04D', '8B4513', 'FFFFFF']},
  'NASADEM 高程'
);
print('GEE 已连接，第一段代码运行成功！');
```



结果说明：运行成功后，Console 中会出现成功提示，地图显示彩色高程。值得注意的是，GEE 中的数据并没有下载到本机，浏览器只收到当前视窗需要的地图瓦片。

1.1.3 为什么选择 GEE?

GEE 平台的建立旨在加速地球科学数据的处理和分析，它提供了大量的卫星图像数据以及丰富的地理空间分析工具，借助 Google 强大的算力，GEE 可以对全球地理数据进行大规模并行处理。GEE 让用户从卫星图像中提取有用信息变得更加容易和高效，在其提供的在线开发环境中，用户可以编写 JavaScript 代码来定义、调用和处理地理数据，让研究者能够集中精力于科研问题本身，而不是数据处理和存储问题。总之，GEE 不仅仅是一个数据仓库，它还是一个交互式地理信息分析工具，集成了先进的图像处理和机器学习算法，使得复杂的数据分析任务在几分钟内即可完成。

1.1.4 常见报错解决

- Image asset ... not found: 数据集 ID 拼写错误或数据已迁移; 从 Data Catalog 复制当前 ID。
- 地图全黑: 显示范围 min/max 不合适; 先查看数据目录的数值范围或用图层设置做拉伸。
- Console 没有内容: 确认点击了 Run, 且代码左侧没有红色语法标记。

1.2 云端计算原理

1.2.1 云端计算原理

GEE Code Editor 是浏览器端开发环境。代码中的 `ee.Image`、`ee.ImageCollection` 等对象是服务器对象: 它们保存的是计算描述, 而不是已经传到浏览器的全部像元。GEE 采用惰性求值; 只有显示图层、打印结果、生成图表或提交导出等请求, 计算图才会被实际求值。

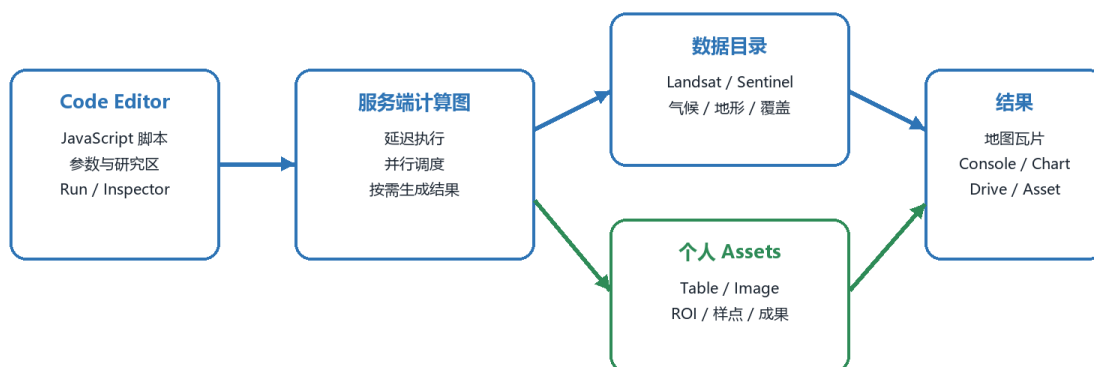
一次典型流程可以理解为:

1. 浏览器创建“筛选 2025 年 Sentinel-2”的计算描述。
2. 追加“去云、算 NDVI、取中位数、裁剪研究区”的操作。
3. 点击 Run 后, 描述被发送给 Earth Engine 服务。
4. 服务器从数据目录读取所需切片, 分布式执行运算。
5. 地图只返回当前视窗瓦片; 统计只返回小字典; 导出任务把完整成果写入 Drive、Cloud Storage 或资产。

“无需下载”并不是“数据永远不能导出”, 而是分析前不必把原始影像全部搬到本地。最终论文图、GeoTIFF、CSV、SHP 仍可按需要导出。

GEE 云端计算: 代码靠近数据

浏览器只提交计算描述; 真正的筛选、合成与统计在 Earth Engine 服务端执行



关键认识 Map.addLayer 显示的是按视窗请求的瓦片; Export 才是持久化结果。

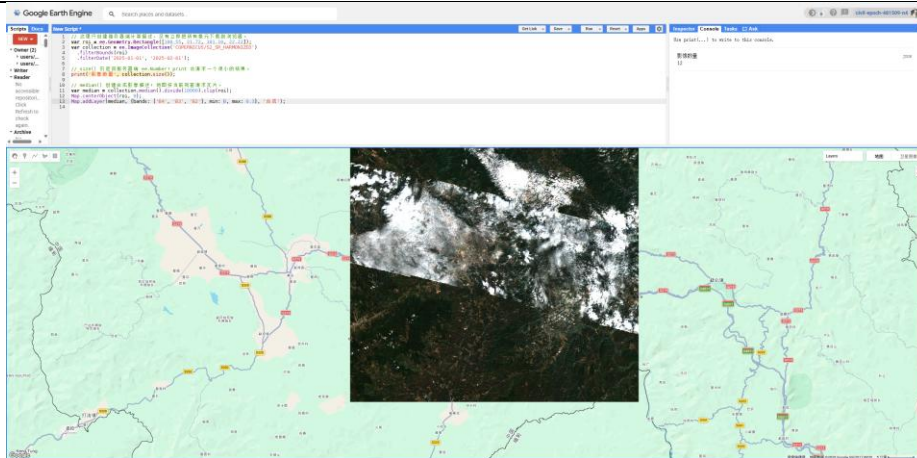
图 1 浏览器提交计算描述, 服务端按需读取公开目录或个人 Assets, 再返回瓦片、统计或导出任务。

示例代码如下：

```
// 这里只创建服务器端计算描述，没有立即把所有像元下载到浏览器。
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2025-01-01', '2025-02-01');

// size() 仍返回服务器端 ee.Number； print 会请求一个很小的结果。
print('影像数量', collection.size());

// median() 创建合成影像描述；地图按当前视窗请求瓦片。
var median = collection.median().divide(10000).clip(roi);
Map.centerObject(roi, 9);
Map.addLayer(median, {bands: ['B4', 'B3', 'B2'], min: 0, max: 0.3}, '合成');
```



结果说明：Console 返回影像数量，地图显示研究区合成。浏览器没有接收完整 Sentinel-2 文件，只获得数量和可视化瓦片。先筛选再计算、尽量在服务器端链式处理、只把小结果传回客户端，是 GEE 高效工作的基本原则。

1.2.2 常见报错解决

- User memory limit exceeded: 缩小研究区、先选波段、提高统计尺度、使用 `tileScale`，不要盲目反复运行。
- Computation timed out: 交互请求太大；将最终计算改为 Export 批任务。
- `getInfo()` 卡住：它会把服务器结果同步拉到客户端；大对象不要用 `getInfo()`。

1.3 账号注册、授权与进入 Code Editor

1.3.1 申请项目

Earth Engine 在 Google Cloud 上运行，需要使用 Cloud 项目进行访问和 API 管理。所有 Earth Engine 调用（无论是来自代码编辑器、客户端库、应用还是 REST API）都会通过云项目进行路由，从而在 Cloud 控制台中实现访问权限控制、资源管理和使用情况监控。

获取 Earth Engine 访问权限

如需使用 Earth Engine，您需要有权访问满足以下条件的 Cloud 项目：

- 已启用 Earth Engine API，
- 已注册用于[商业](#)或[非商业](#)用途，并且
- 授予您（或用户）[正确的角色和权限](#)。

创建项目的教程见官方入口：[Earth Engine Access](#)

Earth Engine 非商业用途层级说明：[Noncommercial Tiers](#)

1.3.2 Code Editor 界面布局介绍

首次访问 Code Editor 时，您可能会看到一个导览，其中重点介绍了 Code Editor 的不同功能。Code Editor 界面布局可以划分为左侧、中间、右侧和地图四个板块。其中，左侧通常包含 Scripts、Docs、Assets 和 AI：

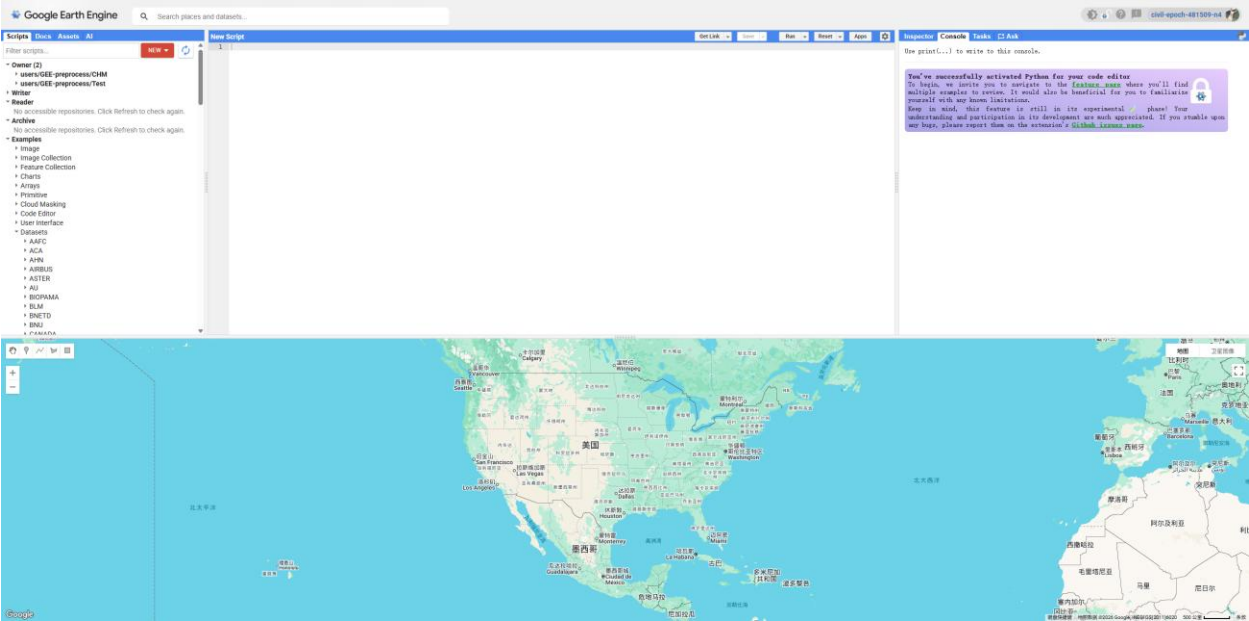
- **Scripts**：基于 Git 的脚本管理器。Owner 保存自己的仓库；Writer/Reader 是他人授权的仓库；Examples 是官方示例。脚本和仓库保留版本历史，可比较或恢复。
- **Docs**：完整 JavaScript API 文档。输入 `normalizedDifference`、`reduceRegion` 等名称可查看参数、返回类型和示例。
- **Assets**：上传和管理栅格、表格等个人资产。上传任务完成后复制完整 Asset ID；脚本共享不等于资产自动共享。
- **AI**：AI 是 Google Earth Engine 官方推出的一个问答系统，你可以向它提问，帮助你编写、调试或理解代码，不过功能比较鸡肋。

中间是 JavaScript 编辑器，提供语法高亮、括号补全、API 自动完成、错误下划线和 Imports 区。GEE Code Editor 目前主要支持 ECMAScript 5 风格；课程统一使用 `var` 和普通 `function`，避免 ES6 语法兼容问题。

地图与右侧面板包括以下内容：

- **Map**：显示影像、矢量和 UI 控件。
- **Layer Manager**：开关图层、调整透明度、改变可视化拉伸和调色板。
- **Inspector**：点击地图查看坐标、缩放级别、像元值和矢量属性。

- **Console:** 显示 `print()`、字典、列表、图表和报错。
- **Tasks:** 显示尚未提交、运行中和已完成的导入/导出任务。
- **Profiler:** 通过 Run 下拉菜单启用，查看算法 CPU 和内存消耗。
- **Ask:** 当前 Code Editor 中的 AI 助手入口；生成代码后仍需人工核对数据集、波段和科研假设。



Earth Engine 代码编辑器（网址为 code.earthengine.google.com）

此外，要介绍一下顶部和地图工具栏，这里用表格进行展示：

控件	功能	新手注意事项
Run	执行当前脚本	Run 下拉菜单可启用 Profiler
Save	保存当前脚本版本	首次保存需选择仓库和文件名
Reset	清空输出地图与 Console，恢复运行前状态	不等于删除脚本
Get Link	生成当前环境快照链接	私有资产不会因链接而自动共享
Get Link 下拉	复制脚本路径、管理历史链接	“快照链接”和“最新保存版本链接”含义不同
Search places and datasets	搜地点、公开栅格和表格数据	从结果页点击 Import 可插入脚本
Project selector	选择执行请求的 Cloud 项目	权限和配额以所选项目为准
Help/Feedback	指南、论坛、导览、快捷键和反馈	报错时先复制完整错误文本
地图点/线/面/矩形工具	交互绘制几何并创建 Imports	点坐标仍是经度在前、纬度在后
地图底图切换	更换地图背景	仅改变背景，不改变分析数据

为了更好地熟悉界面，这里进行一个界面练习，请在脚本编辑界面键入如下代码：

```
print('在 Console 找到这句话');

var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);

Map.centerObject(roi, 9);

Map.addLayer(roi, {color: 'yellow'}, '在 Layer Manager 找到这个图层');
```

运行后依次完成如下操作：在 Console 展开输出；在 Layer Manager 开关图层；启用 Inspector 点击矩形边界；打开 Docs 搜索 Map.addLayer；在 Scripts 新建并保存脚本。

此外，我们再进行一个准入测试，使用如下命令创建一个新脚本：

```
// 测试 1：请求服务器端当前时间。

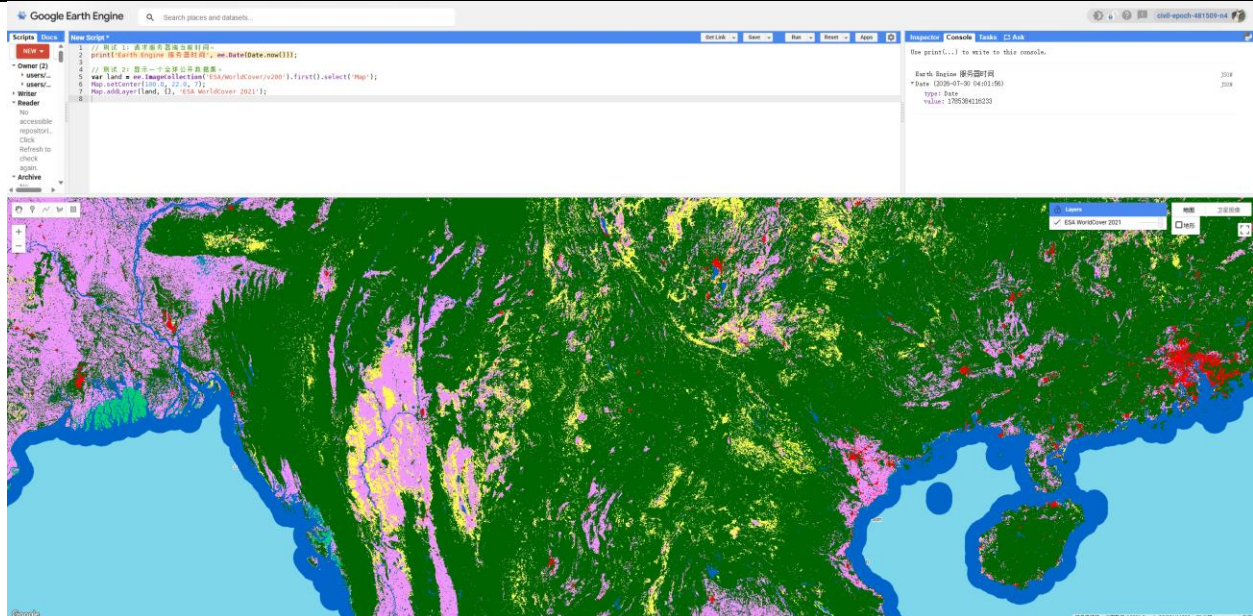
print('Earth Engine 服务器时间', ee.Date(Date.now()));

// 测试 2：显示一个全球公开数据集。

var land = ee.ImageCollection('ESA/WorldCover/v200').first().select('Map');

Map.setCenter(100.8, 22.0, 7);

Map.addLayer(land, {}, 'ESA WorldCover 2021');
```



结果说明：测试代码不需要个人资产。Console 能展开 ee.Date，地图能显示 WorldCover，即说明账号、Cloud 项目、API 和 Code Editor 基本链路正常。

1.3.3 常见报错解决

- **Please select a Cloud Project:** 在 Code Editor 项目选择器中选择已注册项目。
- **Project is not registered for Earth Engine:** 回到 Earth Engine 配置页完成注册和用途选择。

- **403 PERMISSION_DENIED**: 确认登录账号对 Cloud 项目有使用权限；组织项目需管理员授权。
- 非商业访问被暂停：检查资格问卷或重新验证状态，并查看项目配额是否耗尽。
- 多账号跳转错误：退出无关 Google 账号或使用独立浏览器用户配置文件重新登录。

1.4 注册失败、权限与账号问题排查

1.4.1 排查顺序

遇到错误时先保存错误全文，再按以下顺序定位：

1. 身份：浏览器当前登录的是哪个 Google 账号？
2. 项目：Code Editor 当前选择了哪个 Cloud 项目？
3. 注册：该项目是否已注册为 Earth Engine 项目？
4. API：Earth Engine API 是否启用？
5. IAM：账号是否有使用项目和 Earth Engine 资源的权限？
6. 配额：非商业月度 EECU 或并发任务限制是否达到？
7. 资产：脚本引用的个人资产是否存在且已共享？

1.4.2 典型问题与解决方案

现象	可能原因	处理方式
Code Editor 打不开或循环登录	多账号/OAuth 状态冲突	使用 Chrome 独立用户配置文件，仅登录目标账号
公开数据也报 403	项目未注册、API 未启用或 IAM 不足	检查项目配置、API 与项目权限
只有个人资产报错	Asset ID 错误或未共享	在 Assets 复制完整 ID，配置读取权限
Export 一直未开始	任务只创建未提交	在 Tasks 点击 Run；关闭页面前提交
任务运行慢	队列、规模或配额限制	查看任务详情；减少规模或等待，不要重复提交同一任务
旧脚本突然无权限	非商业验证到期或项目切换	重新验证并确认 Code Editor 当前项目

1.4.3 特别提示

阅读 Earth Engine 使用入门，使用 Earth Engine JavaScript API 进行编程，最重要的方法是会查阅官方文档（<https://developers.google.com/earth-engine/guides/>），Earth Engine 文档面向熟悉地理空间数据分析的用户。这些文档主要按数据类型进行构建。例如，左侧导航栏包含指向与重要数据类型（例如 Image、ImageCollection、Feature、FeatureCollection、Geometry、Reducer、Chart、Join 和 Array）相关的部分或页面的链接。本指南仅介绍了这些类型的相关信息，仅供您参考。还有一些部分介绍了机器学习、专用算法或特定于传感器的算法（例如 Landsat 算法）、面向公众的应用、数据（资产）管理，以及 Earth Engine 内部运作的重要详细信息。

1.5 学习路线与核心数据类型概览

1.5.1 推荐学习路线

1. 第一阶段：界面、print、Geometry、Map.addLayer。
2. 第二阶段：Feature/FeatureCollection、Image/ImageCollection。
3. 第三阶段：filter、map、mask、指数、合成、reduce。
4. 第四阶段：图表、导出、任务和性能优化。
5. 第五阶段：科研设计、可比性、趋势、验证、不确定性和复现。

本课程按五类教学：Geometry、Feature、FeatureCollection、Image、ImageCollection。FeatureCollection 是矢量集合，ImageCollection 是栅格时序集合；二者的 filter() 与 map() 形式相似，但元素类型不同。

1.5.2 一次看懂五类对象

```
// Geometry: 空间形状。
var geometry = ee.Geometry.Point([100.80, 22.00]);

// Feature: 空间形状 + 属性。
var feature = ee.Feature(geometry, {name: '样地 1', type: 'forest'});

// FeatureCollection: 多个 Feature。
var featureCollection = ee.FeatureCollection([feature]);

// Image: 一张栅格影像。
var image = ee.Image('NASA/NASADEM_HGT/001').select('elevation');

// ImageCollection: 多张影像组成的集合。
var imageCollection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterDate('2025-01-01', '2025-02-01');

print('Geometry', geometry);
print('Feature', feature);
```

```
print('FeatureCollection', featureCollection);  
print('Image', image);  
print('ImageCollection', imageCollection);
```

在 Console 展开对象，重点观察 Geometry 的坐标、Feature 的 properties、Image 的 bands、ImageCollection 的 features/影像元数据。“单个还是集合”“矢量还是栅格”是识别对象的两个问题。能准确回答，绝大多数类型错误都能避免。

1.5.3 常见报错解决

- ... is not a function: 常见于把 Image 当 ImageCollection 使用；先 print() 检查类型。
- .first() 后无法直接显示：用 ee.Image(collection.first()) 明确转换，尤其在复杂表达式中。
- Feature 没有波段：Feature 是矢量对象，不能使用 select('B4') 读取影像波段。

2 GEE Code Editor 教学

2.1 print、点线面矩形与 Map.addLayer

2.1.1 知识点讲解

print() 用于检查文字、数字、Earth Engine 对象和图表。几何坐标使用 WGS84 经度/纬度时顺序为 [longitude, latitude]。Map.addLayer(object, visParams, name, shown, opacity) 将对象加入地图；矢量可直接添加，也可先 style() 获得更细致样式。

2.1.2 完整可运行代码

```
// =====  
// GEE 从零入门课程：基础语法、print()、几何与矢量可视化  
// 运行环境：https://code.earthengine.google.com/  
// =====  
  
// 1. print(): 把文字、数字和 Earth Engine 对象输出到 Console。  
print('你好， Google Earth Engine! ');  
print('1 + 2 =', 1 + 2);  
  
// 2. 创建一个点。坐标顺序固定为 [经度, 纬度]。  
var point = ee.Geometry.Point([100.80, 22.00]);  
  
// 3. 创建一条线。数组中的每一项都是一个 [经度, 纬度] 坐标。  
var line = ee.Geometry.LineString([  
  [100.62, 21.88],  
  [100.80, 22.00],  
  [101.02, 22.14]  
]);  
  
// 4. 创建一个面。首尾坐标相同，用于闭合多边形。  
var polygon = ee.Geometry.Polygon([[  
  [100.58, 21.76],  
  [101.05, 21.76],  
  [101.05, 22.18],  
  [100.58, 22.18],  
  [100.58, 21.76]  
]]);  
  
// 5. 创建一个矩形，参数依次为西、南、东、北。  
var rectangle = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);  
  
// 6. 把几何对象显示到地图。color 可写颜色名称或十六进制颜色。  
Map.addLayer(point, {color: 'red'}, '点');  
Map.addLayer(line, {color: 'yellow'}, '线');  
Map.addLayer(polygon, {color: '00FFFF'}, '面');  
Map.addLayer(rectangle, {color: 'FFFFFF'}, '矩形');  
  
// 7. 让地图自动缩放到矩形范围；数字 10 是可选缩放级别。  
Map.centerObject(rectangle, 10);  
  
// 8. 计算几何属性。maxError 用 1 米，结果转换为更直观的单位。  
print('点坐标', point.coordinates());  
print('线长度（千米）', line.length({maxError: 1}).divide(1000));  
print('多边形面积（平方千米）',  
  polygon.area({maxError: 1}).divide(1e6));  
  
// 9. 把面包装为 Feature，并添加属性。  
var forestPlot = ee.Feature(polygon, {  
  plot_id: 'P01',
```

```

    land_type: '教学样地',
    survey_year: 2026
  });

// 10. Feature 可用 style() 设置填充色、边线颜色和线宽。
Map.addLayer(forestPlot, {
  color: '006400',
  fillColor: '66AA6666',
  width: 2
}, '带属性的样地');
print('样地 Feature', forestPlot);

```

分析代码中函数的功能：

1. `ee.Geometry.Point` 接收一对坐标。
2. `LineString` 接收有顺序的坐标列表。
3. `Polygon` 外层数组表示环；首尾相同便于直观看出闭合。
4. `Rectangle` 按西、南、东、北定义。
5. `Feature(geometry, properties)` 把空间和调查属性组合起来。
6. 面积默认平方米，除以 `1e6` 转为平方千米。

结果说明：地图出现点、线、面、矩形和样地样式层；Console 给出点坐标、线长度、多边形面积与 `Feature` 属性。`Geometry` 负责“在哪里”，`Feature` 负责“在哪里 + 是什么”。`Map.addLayer` 只控制显示，不会改变原始对象。

2.2 五类核心数据结构

2.2.1 知识点讲解

`Geometry` 是几何；`Feature` 是单个矢量要素；`FeatureCollection` 是矢量集合；`Image` 是多波段栅格；`ImageCollection` 是影像序列。集合的常用思路是先 `filter`，再 `map`，最后用 `first/median/max/reduce` 得到具体结果。

```

// =====
// GEE 从零入门课程：Geometry、Feature、FeatureCollection、
// Image 与 ImageCollection 完整示例
// =====

// ----- 1. Geometry：只有空间形状，没有业务属性 -----
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
Map.centerObject(roi, 9);
Map.addLayer(roi, {color: 'FFFFFF'}, '研究区');
print('Geometry 类型', roi);

// ----- 2. Feature：Geometry + 属性字典 -----
var plotA = ee.Feature(
  ee.Geometry.Point([100.72, 21.92]),
  {plot_id: 'A', forest_type: '常绿阔叶林', canopy: 0.82}
);
var plotB = ee.Feature(
  ee.Geometry.Point([100.91, 22.05]),
  {plot_id: 'B', forest_type: '人工林', canopy: 0.64}
);
print('Feature A', plotA);

```

```

print('A 的林型属性', plotA.get('forest_type'));

// ----- 3. FeatureCollection: 多个 Feature 的集合 -----
var plots = ee.FeatureCollection([plotA, plotB]);
Map.addLayer(plots.style({color: 'FF00FF', pointSize: 6}), {}, '样地点');
print('FeatureCollection', plots);
print('样地数量', plots.size());
print('平均郁闭度', plots.aggregate_mean('canopy'));

// 属性筛选: 保留 canopy 大于 0.7 的样地。
var densePlots = plots.filter(ee.Filter.gt('canopy', 0.7));
print('高郁闭度样地', densePlots);

// ----- 4. Image: 具有一个或多个波段的栅格影像 -----
// 从 Sentinel-2 地表反射率集合中筛选并取得第一景影像。
var s2One = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2025-01-01', '2025-04-01')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
  .sort('CLOUDY_PIXEL_PERCENTAGE')
  .first();

// 打印波段和元数据: 影像波段值按 10000 缩放。
print('单张 Sentinel-2 Image', s2One);
print('单张影像波段', s2One.bandNames());
print('影像日期', ee.Date(s2One.get('system:time_start')));

// 真彩色显示: 红、绿、蓝分别对应 B4、B3、B2。
Map.addLayer(
  s2One.divide(10000).clip(roi),
  {bands: ['B4', 'B3', 'B2'], min: 0, max: 0.3},
  '单景 Sentinel-2 真彩色'
);

// ----- 5. ImageCollection: 具有相同意义的一组时序影像 -----
var s2Collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2025-01-01', '2026-01-01')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 30));

print('Sentinel-2 ImageCollection', s2Collection);
print('影像数量', s2Collection.size());
print('最小云量', s2Collection.aggregate_min('CLOUDY_PIXEL_PERCENTAGE'));

// median() 沿时间维逐像元取中位数, 得到一张合成 Image。
var medianImage = s2Collection.median().divide(10000).clip(roi);
Map.addLayer(
  medianImage,
  {bands: ['B8', 'B4', 'B3'], min: 0, max: 0.45},
  '年度中位数假彩色'
);

```

逐行阅读重点:

get() 读取 Feature 或 Image 的单个属性。

aggregate_mean() 对 FeatureCollection 属性列汇总。

first() 从 ImageCollection 取单景, 仍可能是空值; 先用 size() 检查。

bandNames() 查看影像波段。

median() 沿时间维逐像元聚合, 把 ImageCollection 变为 Image。

结果说明：Console 显示五类对象结构、样地数量、平均郁闭度、单景波段和集合影像数；地图显示 ROI、样地点、单景真彩色和年度中位数假彩色。类型决定可调用的方法。遇到方法错误时，先打印对象，不要靠猜测修改。

2.2.2 常见报错解决

- 单景显示报 null：筛选条件无结果；打印 size()，放宽日期或云量。
- Sentinel-2 显示过亮：原始 SR 值需乘 0.0001 或除以 10000。
- aggregate_mean 返回空：属性不存在、拼写错误或值不是数值。

2.3 筛选、去云、镶嵌与裁剪

2.3.1 知识点讲解

影像筛选一般按范围、时间和元数据属性进行：

- filterBounds(roi)：保留与研究区相交的影像。
- filterDate(start, end)：开始包含、结束不包含。
- filter(ee.Filter.lt(property, value))：按整景云量等属性初筛。

整景云量筛选不能替代逐像元去云。课程用 Sentinel-2 SCL 去除云影、中高概率云、卷云和雪冰。mosaic() 是按集合顺序覆盖，适合同期拼接；median() 对时序取中位数，通常更能抑制残云；clip() 只保留研究区显示/输出范围。

本章与指数章节共用以下脚本，先重点阅读第 1-5 段。

```
// =====
// GEE 从零入门课程：筛选、去云、镶嵌、裁剪与常用指数
// 数据：Sentinel-2 SR Harmonized
// =====

// 1. 定义研究区与分析时段。
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var startDate = '2025-01-01';
var endDate = '2026-01-01'; // filterDate 的结束日期不包含当天。

// 2. SCL 去云函数。
function maskS2(image) {
  // SCL 是场景分类波段；下列类别需要剔除。
  var scl = image.select('SCL');
  var clearMask = scl.neq(0) // 0: 无数据
    .and(scl.neq(1)) // 1: 饱和或坏像元
    .and(scl.neq(3)) // 3: 云影
    .and(scl.neq(8)) // 8: 中概率云
    .and(scl.neq(9)) // 9: 高概率云
    .and(scl.neq(10)) // 10: 卷云
    .and(scl.neq(11)); // 11: 雪/冰

  // 选出常用波段、应用掩膜、缩放为 0—1 反射率，并复制时间属性。
  return image
    .updateMask(clearMask)
    .select(['B2', 'B3', 'B4', 'B8', 'B11'],
      ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0001)
    .copyProperties(image, ['system:time_start']);
}
```

```

}

// 3. 依次执行范围、时间、整景云量筛选，再做逐像元去云。
var s2 = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate(startDate, endDate)
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 40))
  .map(maskS2);

print('去云后的影像集合', s2);
print('筛选后的影像数量', s2.size());

// 4. mosaic() 按集合顺序覆盖；适合同一天或人为排序后的拼接。
var mosaicImage = s2.sort('system:time_start').mosaic().clip(roi);
Map.addLayer(
  mosaicImage,
  {bands: ['red', 'green', 'blue'], min: 0, max: 0.3},
  'mosaic 镶嵌',
  false
);

// 5. median() 对时序逐像元取中位数，通常比直接 mosaic 更稳健。
var composite = s2.median().clip(roi);
Map.centerObject(roi, 9);
Map.addLayer(
  composite,
  {bands: ['red', 'green', 'blue'], min: 0.02, max: 0.30},
  '中位数真彩色'
);

// 6. NDVI = (NIR - Red) / (NIR + Red)。
var ndvi = composite.normalizedDifference(['nir', 'red']).rename('NDVI');

// 7. EVI = 2.5*(NIR-Red)/(NIR+6*Red-7.5*Blue+1)。
var evi = composite.expression(
  '2.5 * (NIR - RED) / (NIR + 6 * RED - 7.5 * BLUE + 1)',
  {
    NIR: composite.select('nir'),
    RED: composite.select('red'),
    BLUE: composite.select('blue')
  }
).rename('EVI');

// 8. NDBI = (SWIR1 - NIR) / (SWIR1 + NIR)。
var ndbi = composite.normalizedDifference(['swir1', 'nir']).rename('NDBI');

// 9. NDWI (McFeeters) = (Green - NIR) / (Green + NIR)。
var ndwi = composite.normalizedDifference(['green', 'nir']).rename('NDWI');

// 10. SAVI = 1.5*(NIR-Red)/(NIR+Red+0.5)，适合稀疏植被。
var savi = composite.expression(
  '1.5 * (NIR - RED) / (NIR + RED + 0.5)',
  {
    NIR: composite.select('nir'),
    RED: composite.select('red')
  }
).rename('SAVI');

// 11. 设置显示参数并逐层添加。
var vegetationPalette = [
  '8c510a', 'd8b365', 'f6e8c3', 'c7eae5', '5ab4ac', '01665e'
];
Map.addLayer(ndvi, {min: -0.2, max: 0.9, palette: vegetationPalette}, 'NDVI');
Map.addLayer(evi, {min: -0.1, max: 0.8, palette: vegetationPalette}, 'EVI', false);
Map.addLayer(ndbi, {min: -0.5, max: 0.5, palette: ['006837', 'FFFFBF', 'A50026']}, 'NDBI', false);
Map.addLayer(ndwi, {min: -0.5, max: 0.5, palette: ['A6611A', 'F5F5F5', '018571']}, 'NDWI', false);
Map.addLayer(savi, {min: -0.1, max: 0.8, palette: vegetationPalette}, 'SAVI', false);

```

```
// 12. 把五个指数合成多波段影像，便于统一统计或导出。
var indices = ndvi.addBands([evi, ndbi, ndwi, savi]);
print('五指数多波段影像', indices);

// 13. 在研究区内计算均值、最小值和最大值。
var reducer = ee.Reducer.mean()
.combine({reducer2: ee.Reducer.min(), sharedInputs: true})
.combine({reducer2: ee.Reducer.max(), sharedInputs: true});

var statistics = indices.reduceRegion({
  reducer: reducer,
  geometry: roi,
  scale: 20,
  maxPixels: 1e9,
  tileScale: 4
});
print('研究区指数统计', statistics);
```

结果说明：Console 返回去云后集合和影像数量；地图默认显示年度中位数真彩色。Layer Manager 中可打开 mosaic 比较，两者可能在残云、接边和日期代表性上不同。标准顺序是“范围/时间/元数据初筛 → 像元质量掩膜 → 波段选择和缩放 → 合成 → 裁剪”。尽早筛选能减少后续计算量。

2.3.2 常见报错解决

- Pattern 'SCL' did not match any bands：使用了不含 SCL 的 Sentinel-2 L1C 集合，改用 COPENICUS/S2_SR_HARMONIZED。
- 去云后全空：阈值过严或研究区云多；逐项暂时关闭掩膜条件定位。
- mosaic 仍有云：mosaic 不自动识别质量；先去云或使用质量镶嵌。

2.4 NDVI、EVI、NDBI、NDWI 与 SAVI

2.4.1 指数原理

NDVI = $(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$ ：绿色植被活力的常用指标，受饱和、土壤背景和大气影响。

EVI = $2.5(\text{NIR} - \text{Red}) / (\text{NIR} + 6\text{Red} - 7.5\text{Blue} + 1)$ ：对高生物量区和大气背景有一定改进。

NDBI = $(\text{SWIR1} - \text{NIR}) / (\text{SWIR1} + \text{NIR})$ ：常用于建成区增强，但裸地也可能响应高。

NDWI = $(\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR})$ ：本课程采用 McFeeters 水体指数；文献中“NDWI”也可能指 Gao 的植被含水指数，必须写清定义。

SAVI = $1.5(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red} + 0.5)$ ：通过土壤调节项减弱稀疏植被的土壤背景影响。

2.4.2 可运行代码

完整代码已在第 9 章脚本中给出。以下是最小指数模板：

```
// composite 的波段已统一命名并缩放为反射率。
var ndvi = composite.normalizedDifference(['nir', 'red']).rename('NDVI');
var evi = composite.expression(
  '2.5 * (NIR - RED) / (NIR + 6 * RED - 7.5 * BLUE + 1)',
  {NIR: composite.select('nir'), RED: composite.select('red'), BLUE: composite.select('blue')}
).rename('EVI');
var ndbi = composite.normalizedDifference(['swir1', 'nir']).rename('NDBI');
var ndwi = composite.normalizedDifference(['green', 'nir']).rename('NDWI');
var savi = composite.expression(
  '1.5 * (NIR - RED) / (NIR + RED + 0.5)',
```

```
{NIR: composite.select('nir'), RED: composite.select('red')}\n).rename('SAVI');
```

结果说明：森林在 NDVI/EVI/SAVI 中通常为较高正值；水体在本课程 NDWI 中通常较高；建成区在 NDBI 中可能较高。具体阈值随传感器、季节、区域和预处理改变，不能照搬。

指数是波段信息的压缩表达，不是地物类别本身。科研写作必须写明公式、波段、数据层级、合成方式和时间窗。

2.4.3 常见报错解决

- 指数范围远超 -1—1：反射率缩放或公式括号错误。
- 波段反了导致森林为负：检查 `normalizedDifference([nir, red])` 的顺序。
- NDWI 结果和论文相反：论文可能使用 NIR/SWIR 定义；先核对公式而不是只看名称。

2.5 均值、最大值、最小值与月度/年度合成

2.5.1 知识点讲解

`ImageCollection.mean()`、`median()`、`min()`、`max()` 沿时间维逐像元聚合。`reduceRegion()` 在空间范围内把一张影像压缩成字典。两者都叫“统计”，但方向不同：前者跨时间，后者跨空间。

月度或年度合成通常用 `ee.List.sequence()` 创建月份/年份，再通过 `map()` 为每个时间段筛选和合成。合成影像应设置 `system:time_start`，否则时序图没有正确横轴。

2.5.2 完整可运行代码

```
// =====\n// GEE 从零入门课程：月度/年度合成、图表与三类导出\n// =====\n\nvar roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);\nMap.centerObject(roi, 9);\n\n// 1. 去云并添加 NDVI.\nfunction prepareS2(image) {\n  var scl = image.select('SCL');\n  var mask = scl.neq(0)\n    .and(scl.neq(1))\n    .and(scl.neq(3))\n    .and(scl.neq(8))\n    .and(scl.neq(9))\n    .and(scl.neq(10))\n    .and(scl.neq(11));\n\n  var reflectance = image.updateMask(mask)\n    .select(['B2', 'B3', 'B4', 'B8', 'B11'],\n      ['blue', 'green', 'red', 'nir', 'swir1'])\n    .multiply(0.0001);\n\n  var ndvi = reflectance\n    .normalizedDifference(['nir', 'red'])\n    .rename('NDVI');\n\n  return reflectance.addBands(ndvi)\n    .copyProperties(image, ['system:time_start']);\n}\n\nvar collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
```

```

.filterBounds(roi)
.filterDate('2024-01-01', '2026-01-01')
.filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 40))
.map(prepareS2);

// 2. 生成 2025 年 12 个月的 NDVI 中位数合成。
var months = ee.List.sequence(1, 12);
var monthlyNdvi = ee.ImageCollection.fromImages(
  months.map(function(month) {
    month = ee.Number(month);
    var start = ee.Date.fromYMD(2025, month, 1);
    var end = start.advance(1, 'month');
    return collection
      .filterDate(start, end)
      .select('NDVI')
      .median()
      .rename('NDVI')
      .set('month', month)
      .set('system:time_start', start.millis());
  })
);
print('2025 月度 NDVI 合成', monthlyNdvi);

// 3. 生成 2024—2025 年年度最大 NDVI 合成。
var years = ee.List.sequence(2024, 2025);
var annualMaxNdvi = ee.ImageCollection.fromImages(
  years.map(function(year) {
    year = ee.Number(year);
    var start = ee.Date.fromYMD(year, 1, 1);
    var end = start.advance(1, 'year');
    return collection
      .filterDate(start, end)
      .select('NDVI')
      .max()
      .rename('NDVI')
      .set('year', year)
      .set('system:time_start', start.millis());
  })
);
print('年度最大 NDVI 合成', annualMaxNdvi);

// 4. 显示 2025 年最大 NDVI。
var ndvi2025 = annualMaxNdvi
  .filter(ee.Filter.eq('year', 2025))
  .first()
  .clip(roi);
Map.addLayer(
  ndvi2025,
  {min: 0.2, max: 0.9, palette: ['F7FCB9', '78C679', '006837']},
  '2025 最大 NDVI'
);

// 5. 研究区月度 NDVI 折线图。
var monthlyChart = ui.Chart.image.series({
  imageCollection: monthlyNdvi,
  region: roi,
  reducer: ee.Reducer.mean(),
  scale: 100,
  xProperty: 'system:time_start'
}).setOptions({
  title: '2025 年研究区月度平均 NDVI',
  hAxis: {title: '月份', format: 'MM'},
  vAxis: {title: 'NDVI', viewWindow: {min: 0, max: 1}},
  lineWidth: 2,
  pointSize: 5,
  colors: ['2E7D32']
});
print(monthlyChart);

```

```

// 6. NDVI 直方图。
var histogram = ui.Chart.image.histogram({
  image: ndvi2025,
  region: roi,
  scale: 100,
  maxBuckets: 40
}).setOptions({
  title: '2025 年最大 NDVI 频率分布',
  hAxis: {title: 'NDVI'},
  vAxis: {title: '像元频数'},
  colors: ['388E3C']
});
print(histogram);
// 7. 导出影像到 Google Drive。运行脚本后到 Tasks 标签点击 Run。
Export.image.toDrive({
  image: ndvi2025.toFloat(),
  description: 'NDVI_2025_Max',
  folder: 'GEE_Course',
  fileNamePrefix: 'ndvi_2025_max',
  region: roi,
  scale: 20,
  maxPixels: 1e13,
  fileFormat: 'GeoTIFF',
  formatOptions: {cloudOptimized: true}
});

// 8. 导出矢量研究区为 SHP。
var roiFeature = ee.Feature(roi, {name: 'Xishuangbanna_demo_roi'});
Export.table.toDrive({
  collection: ee.FeatureCollection([roiFeature]),
  description: 'ROI_Vector',
  folder: 'GEE_Course',
  fileNamePrefix: 'roi_vector',
  fileFormat: 'SHP'
});

// 9. 把年度区域均值转换为表格，再导出 CSV。
var annualTable = annualMaxNdvi.map(function(image) {
  var meanNdvi = image.reduceRegion({
    reducer: ee.Reducer.mean(),
    geometry: roi,
    scale: 100,
    maxPixels: 1e9,
    tileScale: 4
  }).get('NDVI');

  return ee.Feature(null, {
    year: image.get('year'),
    mean_ndvi: meanNdvi
  });
});
print('年度 NDVI 表格', annualTable);

Export.table.toDrive({
  collection: annualTable,
  description: 'Annual_NDVI_Table',
  folder: 'GEE_Course',
  fileNamePrefix: 'annual_ndvi_table',
  fileFormat: 'CSV',
  selectors: ['year', 'mean_ndvi']
});

```

2.5.3 逐行阅读重点

1. `ee.ImageCollection.fromImages()` 把服务器端影像列表变成集合。
2. `start.advance(1, 'month')` 安全处理不同月份天数。

3. 年度最大 NDVI 是每个像元在该年可用观测中的最大值，不一定来自同一日期。
4. 区域统计必须明确 **scale**；合成影像若不指定尺度，默认投影可能带来意外结果。

结果说明：Console 出现 12 景月度合成、2 景年度合成和年度统计表；地图显示 2025 最大 NDVI；Console 还显示月度折线图和直方图；Tasks 创建三类导出任务。

时间合成解决“每个时间段一张代表影像”，空间统计解决“每个区域一个或多个数值”。科研分析通常需要两者结合。

2.5.4 常见报错解决

- 某月影像为空：云多或筛选太严；先统计每月 `size()`，必要时延长窗口。
- 年度最大值异常偏高：可能残云、雪或边缘噪声；改进质量掩膜并检查 Inspector。
- Too many pixels：提高 `scale`、增加 `maxPixels` 或缩小区域；不要无原则把 `maxPixels` 调到极大。

2.6 图层配色、可视化与 Inspector 判读

2.6.1 知识点讲解

可视化参数不会修改分析值。常用参数包括 `bands`、`min`、`max`、`palette`、`gamma` 和 `opacity`。RGB 影像用三个波段；单波段指数用连续调色板。变化图使用以零为中心的发散色带，类别图使用离散色。推荐配色原则：

1. NDVI：低值棕黄、高值绿色。
2. 趋势：退化红、零值浅色、改善绿。
3. 水体：深蓝或青色，但不要把“蓝色=水”当分析结论。
4. 色盲友好：避免只靠红绿区分，增加图例文字和类别编码。

2.6.2 可运行代码

```
var ndvi = ee.ImageCollection('MODIS/061/MOD13A2')
  .filterDate('2025-01-01', '2025-01-17')
  .first()
  .select('NDVI')
  .multiply(0.0001);
var vis = {
  min: -0.2,
  max: 0.9,
  palette: ['8C510A', 'D8B365', 'F6E8C3', '5AB4AC', '01665E']
};
Map.setCenter(100.8, 22.0, 7);
Map.addLayer(ndvi, vis, 'MODIS NDVI');
print('打开 Inspector 后点击地图查看原始 NDVI 值');
```

结果说明：地图显示 NDVI 连续色带。打开 Inspector，点击不同地物，比较像元原值与颜色；修改 `min/max` 后颜色会变化，但 Inspector 数值不变。分析值、显示拉伸和最终制图是三个层次。论文必须说明分析阈值，不能从屏幕颜色反推数值。

2.6.3 常见报错解决

- 调色板顺序反了：交换 palette 顺序，不要把结果乘以 -1 来“修图”。
- 图层颜色一片：min/max 范围过宽或过窄；先看直方图。
- Inspector 显示 masked：该像元被去云/森林掩膜剔除，或当前图层分辨率不同。

2.7 ui.Chart 图表绘制

2.7.1 知识点讲解

ui.Chart.image.series() 对单个区域绘制 ImageCollection 时间序列；seriesByRegion() 比较多个分区；ui.Chart.image.histogram() 查看单张影像的值分布；ui.Chart.feature.groups() 对长表按分类字段分组作图。图表通过 print(chart) 显示在 Console。

2.7.2 可运行代码

```
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var modis = ee.ImageCollection('MODIS/061/MOD13A2')
  .filterBounds(roi)
  .filterDate('2024-01-01', '2026-01-01')
  .select('NDVI')
  .map(function(image) {
    return image.multiply(0.0001)
      .rename('NDVI')
      .copyProperties(image, ['system:time_start']);
  });

var chart = ui.Chart.image.series({
  imageCollection: modis,
  region: roi,
  reducer: ee.Reducer.mean(),
  scale: 1000,
  xProperty: 'system:time_start'
}).setOptions({
  title: '研究区 MODIS NDVI 时间序列',
  hAxis: {title: '日期'},
  vAxis: {title: 'NDVI'},
  lineWidth: 1,
  pointSize: 2,
  colors: ['2E7D32']
});

print(chart);
```

结果说明：Console 显示两年 NDVI 折线。季节性植被会形成周期波动；异常低值可能是物候、干旱、扰动或残余质量问题，需结合影像和质量波段解释。图表是质量控制工具，也是科研结果表达工具。先用图表检查时序，再做趋势分析。

2.7.3 常见报错解决

- 图表没有日期：映射函数丢失 system:time_start；使用 copyProperties。
- 图表过慢：区域过大且 scale 太细；图表只用于概览时提高 scale。
- 折线断裂：某些时点区域全部被掩膜；检查云掩膜与数据覆盖。

2.8 导出影像、矢量和表格

2.8.1 知识点讲解

`Export.image.toDrive()` 导出栅格，`Export.table.toDrive()` 同时用于矢量和属性表。代码运行只创建未提交任务；必须在 Tasks 点击 Run。任务提交后关闭页面不会丢失，未提交任务则会随页面关闭丢失。影像导出关键参数：

- **image**: 待导出影像。
- **region**: 导出范围。
- **scale**: 名义像元大小，不能把 30 m Landsat 设为 10 m 就当成真实 10 m 信息。
- **crs/crsTransform**: 论文需要严格网格对齐时显式指定。
- **maxPixels**: 允许的最大像元数。

formatOptions.cloudOptimized: 生成 Cloud Optimized GeoTIFF。

表格导出关键参数：**collection**、**fileFormat**、**selectors**。Shapefile 字段名和字符编码限制较多；复杂属性优先 GeoJSON/CSV，空间分析优先 GeoPackage（需在本地转换）或 Earth Engine Asset。

2.8.2 可运行代码

完整示例见第 11 章脚本的第 7-9 段。最小模板如下：

```
Export.image.toDrive({
  image: ndvi2025.toFloat(),
  description: 'NDVI_2025_Max',
  folder: 'GEE_Course',
  region: roi,
  scale: 20,
  maxPixels: 1e13,
  fileFormat: 'GeoTIFF',
  formatOptions: {cloudOptimized: true}
});

Export.table.toDrive({
  collection: annualTable,
  description: 'Annual_NDVI_Table',
  folder: 'GEE_Course',
  fileFormat: 'CSV',
  selectors: ['year', 'mean_ndvi']
});
```

结果说明：Tasks 中出现任务。点击 Run，核对文件名、Drive 文件夹、范围、分辨率和 CRS，再提交。完成后到 Google Drive 指定文件夹查看。显示正确不等于导出正确。提交前检查单位、数据类型、掩膜、范围、分辨率、CRS、NoData 和波段名。

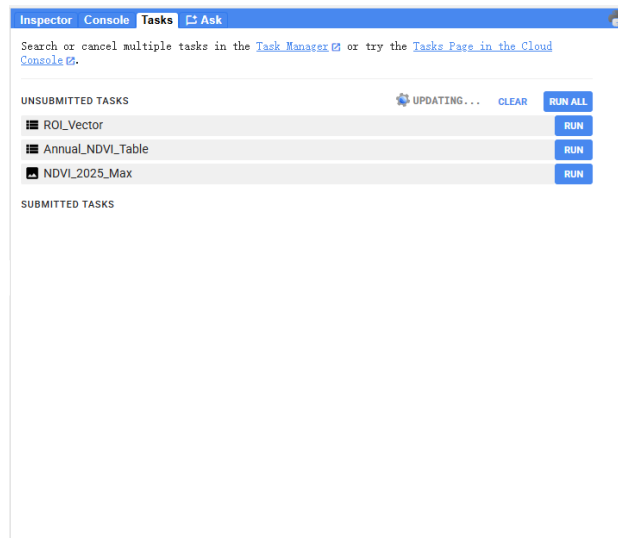
2.8.3 导出技巧 - Google Earth Engine 一键 RUN

在批量下载数据的时候，会有很多 run 的按钮，需要一个一个去点，很麻烦，类似于下图：



使用国外作者制作的一个谷歌浏览器的插件：Open Earth Engine extension，可以一键导出所有需要下载的文件。[下载链接](#)

当安装好该插件后刷新 GEE 页面，即可在 Task 界面看到 RUN ALL 按钮，非常方便有效。



2.8.4 常见报错解决

- Export too large: 缩小区域、提高 scale、分区导出或减少波段。
- 输出影像偏移: 为严格对齐使用参考影像的 crs 和 crsTransform，不要只依赖 scale。
- CSV 少了几何: CSV 默认主要是属性；需要空间可导出 GeoJSON/SHP，或在 selectors 加 .geo（格式支持时）。

2.9 客户端/服务器、map 与性能调试

2.9.1 知识点讲解

普通 JavaScript 值（字符串、数组、数字）在客户端；ee.* 对象在服务器。服务器端 map(function(element){...}) 中不能使用客户端 if 读取服务器属性，也不要再映射函数中调用 print() 或 getInfo()。

性能优化顺序：

1. 尽早 filterBounds/filterDate/select。
2. 不重复计算同一复杂表达式；保存中间对象描述。
3. 交互统计提高 scale；最终导出再用目标尺度。
4. reduceRegion/reduceRegions 明确 scale、maxPixels，内存不足时尝试 tileSize: 2 或 4。
5. 使用 Profiler 找到最耗时的算法，而不是凭感觉重写全部代码。

2.9.2 错误与正确示例

```
var years = ee.List.sequence(2020, 2025);

// 正确：在服务器端 map 中把元素显式转换为 ee.Number。
var labels = years.map(function(year) {
  year = ee.Number(year);
  return ee.String('year_').cat(year.format('%d'));
});
print('服务器端年份标签', labels);

// 客户端固定参数可使用普通 if；完整项目用它切换 S2/Landsat。
var source = 'S2';
if (source === 'S2') {
  print('使用 Sentinel-2');
} else {
  print('使用 Landsat');
}
```

结果说明：Console 返回服务器端列表和数据源文字。尝试把 year 直接与客户端字符串做不支持的运算，会出现类型或序列化错误。

GEE 编程的难点不是 JavaScript 语法本身，而是理解“计算在哪里执行”。链式服务器对象是主角，客户端只负责组织参数和界面。

2.9.3 常见报错解决

- A mapped function's arguments cannot be used in client-side operations：把客户端运算改为 ee.Number/ee.String/ee.Algorithms.If 等服务器表达式。
- User memory limit exceeded：提高 tileSize、减少波段/区域/时段，或改用批导出。
- Too many concurrent aggregations：减少在 map 内同时执行的大量 reduce，分批计算。

2.10 综合练习

2.10.1 任务

以自己的研究区完成以下操作：

1. 绘制或定义一个 ROI。
2. 获取 2025 年 Sentinel-2 SR Harmonized，整景云量小于 30%。
3. SCL 去云，计算 NDVI 和 NDWI。
4. 生成中位数合成和最大 NDVI。

5. 绘制月度 NDVI 区域均值图。
6. 导出最大 NDVI GeoTIFF 和月度均值 CSV。

2.10.2 验收标准

1. Console 中影像数大于 0。
2. 波段名称和缩放正确，NDVI 大体在 -1—1。
3. 时间图横轴是日期且没有错误的 1970 年。
4. Tasks 中影像和表格任务参数完整。
5. 代码包含数据集 ID、ROI、时间、云量阈值、scale 和导出范围。

2.10.3 常见报错解决

- 替换 ROI 后超时：先用较小 ROI 和 100 m 统计尺度验证逻辑。
- 自己的矢量资产无权限：Assets 中复制 ID，并确认当前账号和 Cloud 项目。
- 月度表有 null：该月没有有效观测；在输出中保留缺测并说明，不建议随意用 0 填补。

3 从个人数据到公开应用与 AI 辅助编程

3.1 上传、检查与管理个人 Asset

3.1.1 Asset 是什么

Asset 是存放在 Earth Engine 中、可由脚本通过稳定 ID 访问的用户数据。常见类型包括 Table（矢量/表格）、Image（栅格）、ImageCollection 和 Folder。Cloud 项目下的典型路径是 projects/项目 ID/assets/资产名。旧教程中常见的 users/用户名/... 属于历史用户根路径；新项目优先使用项目级路径，并以 Assets 面板实际显示的 ID 为准。

上传是异步“摄取”（ingestion）：文件先被传送，随后 Earth Engine 建立空间索引、金字塔与元数据。只有 Tasks 显示 COMPLETED 后，脚本才能稳定读取。

资产名称、权限、配额和摄取状态是四个不同问题。

个人 Asset 上传闭环

上传不是复制文件：它会创建异步 ingestion task，成功后才得到可引用的 Asset ID

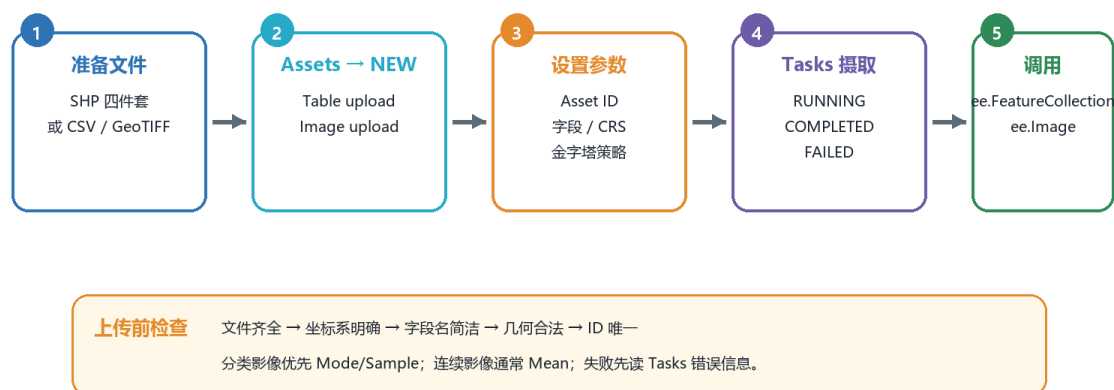
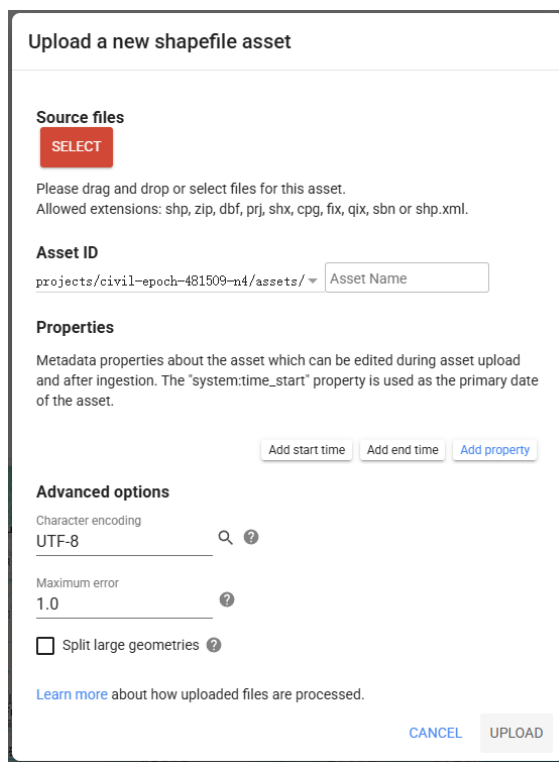
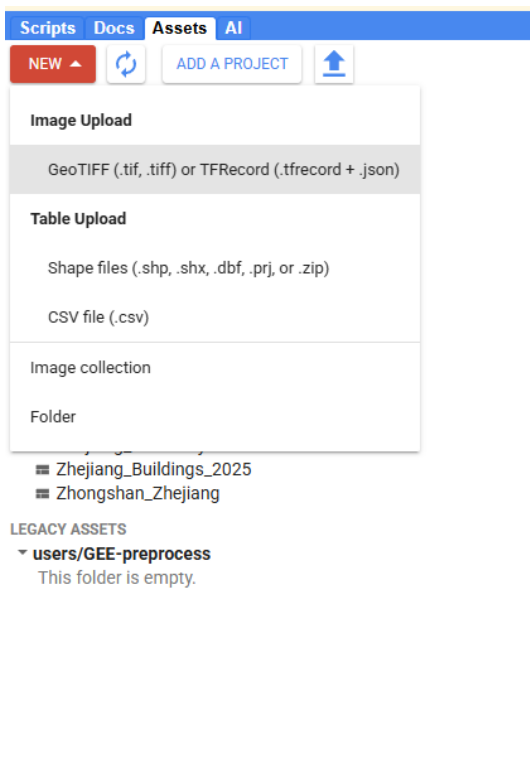


图3 个人 Asset 上传闭环。Table 与 Image 都需要在 Tasks 中完成摄取，随后才获得可用于 ee.FeatureCollection 或 ee.Image 的 Asset ID。

3.1.2 上传 Shapefile 或 CSV 为 Table Asset

1. 在本地先检查要上传的文件的坐标系、几何合法性、字段名与编码。建议先重投影到 EPSG:4326，并避免超长、重复或包含特殊字符的字段名。
2. Shapefile 至少同时选择 .shp、.dbf、.shx，有 .prj 时一并上传，总之就是能上传的都是上传。
3. 打开 Code Editor 左侧 Assets，点击 NEW，选择 Table upload。

4. 选择文件，填写唯一的 Asset ID 和说明。ID 建议仅用英文字母、数字与下划线。
5. CSV 若只有普通字段，需要指定经纬度列；若有 WKT 或 GeoJSON 几何列，按上传对话框选择对应方式。
6. 点击 Upload 后到 Tasks 查看状态。失败时展开完整错误，不要重复提交同一错误文件。
7. 成功后在 Assets 中点击资产，检查类型、字段、要素数、范围和 ID；点击 Import 可自动插入 Imports，也可手写完整 ID。



最小读取与自检代码：

```
// 替换为 Assets 面板复制的完整 ID。
var table = ee.FeatureCollection('projects/your-project/assets/forest_roi_2026');

// 先检查再参与分析；first() 用于查看属性和值。
print('资产对象', table);
print('要素数量', table.size());
print('首个要素', table.first());
print('字段名', ee.Feature(table.first()).propertyNames());

Map.centerObject(table, 8);
Map.addLayer(table.style({
  color: '00FFFF',
  fillColor: '00000000',
  width: 2
}), {}, '个人 Table Asset');
```

3.1.3 上传 GeoTIFF 为 Image Asset

1. 检查 GeoTIFF 的 CRS、仿射变换、波段数、数据类型、NoData 与单位。地理参考必须存在且正确。
2. Assets → NEW → Image upload, 选择一个或多个兼容切片; Code Editor 上传单个 GeoTIFF 的官方说明上限为 10 GB, 超大数据应改用 Cloud Storage/命令行摄取方案。
3. 为连续变量选择适合的金字塔策略, 默认 Mean 常用于连续影像; 土地覆盖等离散分类宜使用 Mode 或 Sample, 避免缩放时生成不存在的类别均值。
4. 对透明区、NoData、时间戳和波段名称作明确设置; 开始摄取后在 Tasks 等待完成。
5. 读取后打印 bandNames()、projection()、nominalScale() 和最小/最大值。

```
var image = ee.Image('projects/your-project/assets/canopy_height_2025');
print('波段', image.bandNames());
print('投影', image.projection());
print('名义分辨率', image.projection().nominalScale());

Map.addLayer(image, {min: 0, max: 40, palette: ['fff7bc', '7fcdbb', '225ea8']},
'个人 Image Asset');
```

3.1.4 Asset 权限与管理

Asset 权限和 Cloud 项目 IAM 相关但不完全相同。Viewer 可读取, Writer 可修改, Admin 可管理权限; 是否能“使用某个 Cloud 项目发起请求”还取决于该项目的 IAM/Service Usage 权限。分享脚本并不会自动分享脚本引用的个人资产。

在 Code Editor/Asset Manager 中选择目标资产后使用 Share:

- 与具体协作者共享: 优先授予最小读取权限, 并定期复核成员。
- 公开只读: 仅在数据许可、隐私和机构政策允许时使用 anyone can read。
- 为 App 提供资产: 按 Apps 发布流程向应用服务账号或应用要求的主体授予读取权限; 发布后用未登录/无 Earth Engine 账号的浏览器窗口验证。
- 删除或改名: 会破坏硬编码 Asset ID; 正式项目先更新脚本、文档和复现实验记录。

官方上传与管理依据: [Managing assets](#)、[Table upload](#)、[Image upload](#)、[Access control](#)。

结果说明: 上传成功不等于数据正确。Table 至少核对要素数、属性、空间范围、几何; Image 至少核对波段、CRS、像元大小、NoData、单位与统计分布。任何一项异常都应回到源数据或上传参数修正。

Asset 的可靠流程是“源文件检查 → 上传参数 → Tasks 摄取 → 结构检查 → 权限 → 脚本引用 → 复现记录”。个人资产路径是外部依赖, 应在 README 记录来源、版本和所需权限。

3.1.5 常见报错解决

- Asset not found: 复制完整 ID; 检查拼写、摄取是否完成、账号和 Cloud 项目。

- Shapefile 上传失败：确认 .shp/.dbf/.shx 同名且齐全，几何合法，压缩包内不要嵌套多层目录。
- 地图出现在错误国家：坐标系声明或经纬度列错误；先在本地图验证，不要靠 GEE 猜 CRS。
- 分类栅格出现小数：金字塔策略使用了 Mean；重新摄取并设为 Mode/Sample。
- 他人能打开脚本但看不到 ROI：缺少 Asset Viewer 权限，或脚本中仍引用旧 ID。

3.2 把个人 Asset 用作 ROI 与分区统计

3.2.1 先区分 FeatureCollection 与 Geometry

个人行政区、保护区或样地通常以 FeatureCollection 读取。它既包含几何，也包含名称、代码、类型等属性。roiFc.geometry() 会把集合几何合并为一个 Geometry，适合 filterBounds、clip、centerObject 和单区 reduceRegion；但逐区 reduceRegions 或导出分区结果需要保留 FeatureCollection。

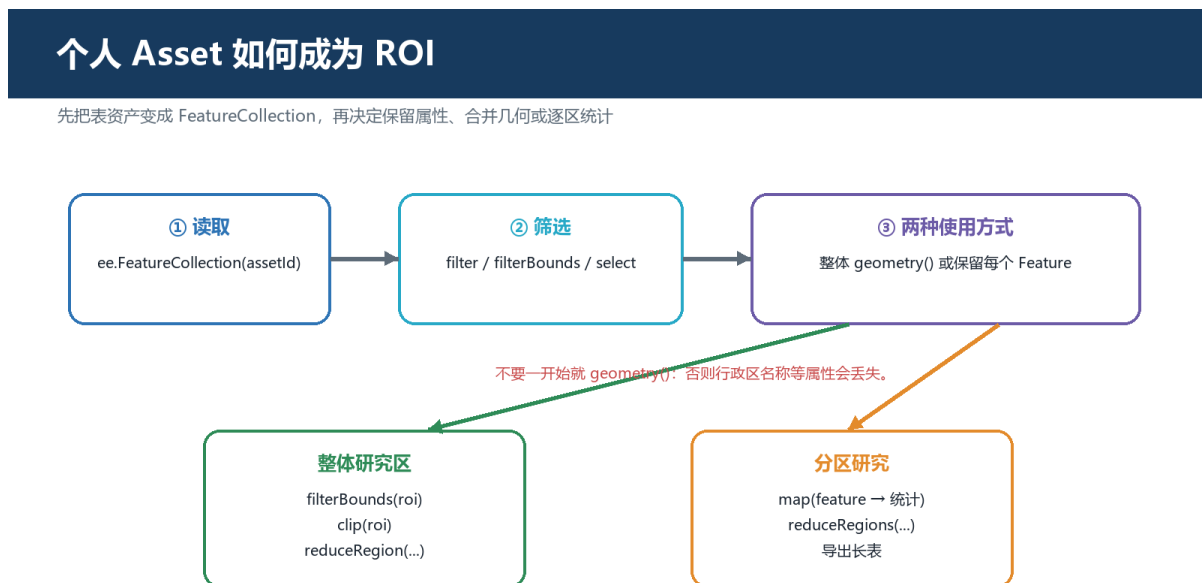


图 4 整体几何用于筛选、裁剪和单区统计，保留 FeatureCollection 用于逐区统计与属性导出。

3.2.2 属性筛选与防空检查

```

var allZones = ee.FeatureCollection('projects/your-project/assets/county_boundary');
print('字段名', ee.Feature(allZones.first()).propertyNames());

// 字段名和值必须与资产真实属性完全一致。
var roiFc = allZones.filter(ee.Filter.eq('NAME', '示例县'));
print('筛选后数量：必须 > 0', roiFc.size());
print('筛选后首个要素', roiFc.first());

var roi = roiFc.geometry();
Map.centerObject(roiFc, 9);
Map.addLayer(roiFc, {color: 'yellow'}, '筛选后的 ROI');

```

在教学调试阶段先看 `size()`；不要立即用 `ee.Image(collection.first())` 或进入长管线，否则空集合错误会在很后面才暴露。字段值可能含空格、大小写、不同语言或前导零，应从 `first()` 和属性直方图读取真实值。

3.2.3 整体 ROI、逐区统计与几何复杂度

```
// 整体 ROI：用于影像空间筛选和裁剪。
var images = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2025-01-01', '2026-01-01');

// 示例影像：正式分析还需按数据目录执行去云与缩放。
var image = images.median().select('B8').multiply(0.0001).rename('nir');

// 单个整体统计。
var whole = image.reduceRegion({
  reducer: ee.Reducer.mean(),
  geometry: roi,
  scale: 20,
  maxPixels: 1e9,
  tileScale: 2
});
print('整体均值', whole);

// 逐区统计：输入仍是带属性的 FeatureCollection。
var byZone = image.reduceRegions({
  collection: allZones,
  reducer: ee.Reducer.mean(),
  scale: 20,
  tileScale: 2
});
print('逐区表', byZone.limit(5));
```

复杂边界会显著增加裁剪和统计开销。`bounds()` 会变成外接矩形，速度快但改变研究区；`simplify(tolerance)` 会近似边界，必须记录容差并评估面积差；不要为了速度无说明地改变几何。优先通过缩小日期、选择必要波段、提高统计 `scale`、分区运行和批导出来控制资源。

3.2.4 完整可运行示例

下面脚本默认使用内置小矩形；把 `USE_PERSONAL_ASSET` 改为 `true` 并替换路径后，即可复用个人 ROI。默认分支不读取占位 ID，因此可直接运行。

```
// =====
// 06 个人 Asset、ROI 与可视化
// 默认模式无需个人 Asset，可直接运行；准备好资产后再切换开关。
// =====

// ----- 1. 研究区来源 -----
// false: 使用北京附近的内置矩形，保证脚本可直接运行。
// true: 读取你自己的 Table Asset；切换前必须替换下一行的 Asset ID。
var USE_PERSONAL_ASSET = false;
var PERSONAL_ASSET_ID = 'projects/your-cloud-project/assets/your_roi';

// 小范围演示 ROI，避免初次运行时计算量过大。
var demoRoiFc = ee.FeatureCollection([
  ee.Feature(
    ee.Geometry.Rectangle([116.05, 39.70, 116.75, 40.25]),
    {roi_name: 'Beijing_demo'}
  )
]);
```

```

// JavaScript 本地布尔值决定读取哪个分支：默认不会访问占位 Asset ID。
var roiFc = USE_PERSONAL_ASSET
  ? ee.FeatureCollection(PERSONAL_ASSET_ID)
  : demoRoiFc;

// geometry() 将集合中全部要素合并成一个分析几何。
// 若要逐行政区统计，请保留 roiFc 的属性，不要只保留 roi。
var roi = roiFc.geometry();

print('ROI FeatureCollection', roiFc);
print('ROI 要素数量', roiFc.size());
print('ROI 首个要素属性', roiFc.first());

// ----- 2. Sentinel-2 去云与缩放 -----
function maskS2Sr(image) {
  var scl = image.select('SCL');
  // 去除云影、云、卷云、雪冰；保留其他地表类别。
  var clear = scl.neq(3)
    .and(scl.neq(8))
    .and(scl.neq(9))
    .and(scl.neq(10))
    .and(scl.neq(11));

  // SR 波段比例因子为 0.0001，并保留时间属性供图表使用。
  return image.updateMask(clear)
    .select(['B2', 'B3', 'B4', 'B8'], ['blue', 'green', 'red', 'nir'])
    .multiply(0.0001)
    .copyProperties(image, ['system:time_start']);
}

var images = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate('2024-04-01', '2024-11-01')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 60))
  .map(maskS2Sr);

print('筛选后影像数量', images.size());

// 中位数合成对残余异常值较稳健；clip 只保留 ROI 范围。
var composite = images.median().clip(roi);
var ndvi = composite.normalizedDifference(['nir', 'red']).rename('NDVI');

// ----- 3. 五种常用可视化 -----
// 真彩色：三个波段分别映射到红、绿、蓝显示通道。
var rgbVis = {
  bands: ['red', 'green', 'blue'],
  min: 0.02,
  max: 0.30,
  gamma: 1.1
};

// 连续变量：固定 min/max，便于不同日期或区域横向比较。
var ndviVis = {
  min: 0,
  max: 0.9,
  palette: ['8c510a', 'f6e8c3', 'c7eae5', '5ab4ac', '01665e']
};

// 分类变量：阈值只用于教学，不等同于通用森林判定标准。
var vegetationClass = ndvi.expression(
  "b('NDVI') < 0.2 ? 0 : b('NDVI') < 0.5 ? 1 : 2"
).rename('vegetation_class').toByte();
var classVis = {
  min: 0,
  max: 2,
  palette: ['d9d9d9', 'fee08b', '1a9850']
};

```

```

// 矢量 style() 会把 FeatureCollection 渲染成便于显示的 Image。
var roiStyle = roiFc.style({
  color: '00FFFF',
  fillColor: '00000000',
  width: 3
});

Map.centerObject(roiFc, 9);
Map.addLayer(composite, rgbVis, 'S2 真彩色');
Map.addLayer(ndvi, ndviVis, 'NDVI 连续色带');
Map.addLayer(vegetationClass, classVis, '植被等级（教学阈值）', false);
Map.addLayer(roiStyle, {}, 'ROI 边界');

// ----- 4. 图例 -----
var legend = ui.Panel({
  style: {position: 'bottom-left', padding: '8px 12px'}
});
legend.add(ui.Label({
  value: 'NDVI 图例',
  style: {fontWeight: 'bold', fontSize: '14px'}
}));

var labels = ['低: 0.0', '中: 0.45', '高: 0.9'];
var colors = ['8c510a', 'f6e8c3', '01665e'];
for (var i = 0; i < labels.length; i++) {
  var colorBox = ui.Label("", {
    backgroundColor: '#' + colors[i],
    padding: '8px',
    margin: '0 6px 4px 0'
  });
  var description = ui.Label(labels[i], {margin: '0 0 4px 0'});
  legend.add(ui.Panel([colorBox, description], ui.Panel.Layout.flow('horizontal')));
}
Map.add(legend);

// ----- 5. Inspector 的统计替代: 直方图 -----
var histogram = ui.Chart.image.histogram({
  image: ndvi,
  region: roi,
  scale: 30,
  maxBuckets: 40
}).setOptions({
  title: 'ROI 内 NDVI 分布',
  hAxis: {title: 'NDVI'},
  vAxis: {title: '像元数'},
  legend: {position: 'none'},
  colors: ['2E8B57']
});
print(histogram);

// ----- 6. 导出 -----
// Run 后到 Tasks 面板手动提交任务。
Export.image.toDrive({
  image: ndvi,
  description: 'ROI_NDVI_2024',
  folder: 'GEE_exports',
  fileNamePrefix: 'ROI_NDVI_2024',
  region: roi,
  scale: 10,
  maxPixels: 1e10
});

// 如需写回 Asset，可取消下段注释并替换 assetId。
// 分类影像通常设置 pyramidingPolicy 为 mode；连续 NDVI 可保留 mean。
// Export.image.toAsset({
//   image: ndvi,
//   description: 'ROI_NDVI_2024_toAsset',

```

```
// assetId: 'projects/your-cloud-project/assets/ROI_NDVI_2024',
// region: roi,
// scale: 10,
// maxPixels: 1e10,
// pyramidingPolicy: {'default': 'mean'}
// });
```

结果说明：地图应出现真彩色、连续 NDVI、可选分类图层和 ROI 边界；Console 显示 ROI 属性、影像数量和 NDVI 直方图；Tasks 生成一个 Drive 导出任务。切换个人 Asset 后，首先比较 `roiFc.size()` 和地图范围是否符合预期。

FeatureCollection 是“带属性的分区”，`geometry()` 是“用于空间运算的整体形状”。同一项目常同时需要二者，变量名应区分为 `roiFc` 和 `roi`。

3.2.5 常见报错解决

- `Collection.geometry: Geometry has too many edges`: 边界过于复杂；评估 `simplify`、分区处理或更粗 `scale`。
- `filterBounds` 结果为 0: 先显示 ROI，检查经纬度、CRS 和时间范围。
- `reduceRegions` 属性丢失: 中途把集合转换成了 `Geometry`，或构建新 `Feature` 时没有复制属性。
- 分区表大量 `null`: 相应区域没有有效像元；检查掩膜、日期、`scale` 与边界重叠。

3.3 建立可解释、可比较的可视化

3.3.1 可视化与分析值分离

`Map.addLayer(image, visParams, name)` 把数值映射到屏幕颜色。`min/max/palette/gamma` 只影响显示，不会修改原影像；`image.visualize()` 才会生成 8-bit RGB 显示影像。统计、趋势和科研导出应使用原始数据，制图导出才考虑 `visualize()`。

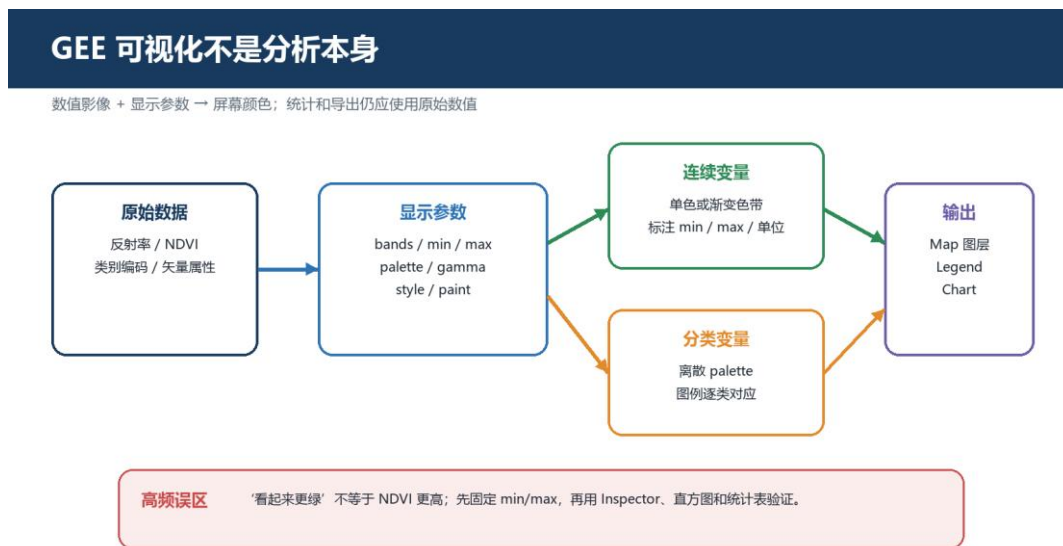


图5 GEE 可视化层次。原始数据经过显示参数形成 RGB、连续或分类图层，但数值统计仍应回到原始影像。

3.3.2 五类常用方法

类型	典型参数/方法	适用场景	验收重点
RGB	bands/min/max/gamma	真彩色、假彩色	波段顺序、缩放后数值范围
单波段连续	min/max/palette	NDVI、温度、坡度	固定范围、单位、色盲友好
分类	整数编码 + 离散 palette	土地覆盖、变化类型	每一类与颜色一一对应
矢量	style()、paint()	ROI、样点、道路	线宽、填充、图层遮挡
图表	ui.Chart	分布、时序、分区比较	reducer、scale、时间属性

比较两个年份时，应固定相同的 min/max/palette；否则自动拉伸会把很小差异夸大。NDVI 理论范围是 -1-1，但显示范围应由研究区分布和表达目的确定；不能为了“更好看”截断异常而不报告。

3.3.3 Inspector、直方图与统计三重检查

1. Inspector：点击至少 5 个不同地物像元，核对显示颜色与数值。
2. 直方图/分位数：检查是否大量堆积在 min/max 外，是否出现不合理尖峰。
3. 区域统计：用明确 reducer、scale、CRS 和 mask 计算均值/面积；与图表设置保持一致。

```
var stats = ndvi.reduceRegion({
  reducer: ee.Reducer.percentile([2, 25, 50, 75, 98]),
  geometry: roi,
  scale: 30,
  maxPixels: 1e8,
  tileScale: 2
});
print('用于确定显示范围的分位数', stats);
```

结果说明：一个“可信图层”应同时回答：颜色代表什么数值、显示范围如何确定、原始数据单位是什么、掩膜在哪里、比较图是否同尺度，以及地图印象能否由 Inspector/直方图/统计支持。

3.3.4 图例原则

图例至少包含变量名、端点值或类别、单位和必要方法说明。连续色带的视觉顺序应与数值顺序一致；改善/退化等有正负方向的变量宜用以 0 为中点的发散色带；分类色带不要使用连续渐变暗示不存在的顺序。

3.3.5 常见报错解决

- 图层全黑/全白：min/max 与缩放后数值不匹配；先打印分位数。
- RGB 偏色：波段顺序错误或未应用比例因子。
- 分类图颜色错位：检查类别最小/最大值与 palette 顺序，打印频数直方图。
- 地图好看但统计异常：显示参数不会修复数据；回查掩膜、单位、scale 和 CRS。

3.4 共享脚本、GitHub、Assets 与公开 Earth Engine App

3.4.1 四种共享方式解决不同问题



图 6 共享方式决策图：脚本协作、GitHub 版本归档、公开 Earth Engine App 与 Asset 数据权限不能互相替代。

方式	适合受众	能分享什么	不能自动解决什么
Get Link / Script repository	已有 GEE 权限的协作者	代码、导入状态、地图视图	私有 Asset 权限、数据许可
GitHub	需要版本、引用和下载的人	.js、README、图、许可、版本历史	在 GEE 中直接运行、个人 Assets
Asset sharing	需要读写数据的人或 App	Table/Image 等资产访问	代码说明、分析复现
Earth Engine App	普通公众和无 GEE 账号读者	可交互地图与 UI	源数据许可、方法透明、无限算力

3.4.2 共享脚本给协作者

1. Save 到明确仓库与文件名，并写头部注释：数据 ID、时间、ROI 来源、作者/版本、运行日期。
2. 使用 Get Link 生成脚本快照或分享脚本仓库；说明链接是固定快照还是会跟随最新版本。
3. 列出脚本依赖的所有个人 Asset ID，并分别授予协作者 Viewer 权限。
4. 让协作者使用自己的 Cloud 项目运行；他需要相应项目和 Earth Engine 访问权限。
5. 用另一账号复测，不要只在作者账号确认。

3.4.3 发布 Earth Engine App

Earth Engine App 由 Code Editor 的 Apps 面板发布到唯一 URL。公开 App 的访客通常不需要 Earth Engine 账号；也可限制给 Google Group。发布过程需要关联已注册的 Cloud 项目。App 使用到的个人 Assets 必须按官方流程共享给 App/公众，否则作者能看、访客看不到。

1. 先把分析重构为 ui.Panel、ui.Map、选择器、按钮、图例和状态提示。
2. 默认显示一个计算量小、含义清楚的结果；不要在页面加载时跑全国多年全分辨率统计。
3. Apps 面板新建 App，选择 Cloud 项目、源脚本和访问方式。
4. 检查每个个人 Asset 的读取权限、第三方数据许可、隐私与配额。
5. 发布后在未登录窗口测试 URL、控件、图层、移动端宽度、错误状态和加载时间。
6. 在 App 中展示数据来源、时间窗口、方法、限制和联系方式；不要把 App 的图层当作论文验证。

官方依据：[Earth Engine Apps](#)。官方特别说明：公开 App 可供无 Earth Engine 账号的用户访问；源 JavaScript 不能视为保密内容，因此不要把口令、密钥或敏感逻辑写入客户端脚本。

3.4.4 完整 App 示例

```
// =====
// 07 可公开访问的 Earth Engine App 教学模板
// 使用公开数据和内置 ROI，不依赖个人 Asset，可直接运行和发布。
// =====

// ----- 1. 应用数据与函数 -----
var roi = ee.Geometry.Rectangle([116.05, 39.70, 116.75, 40.25]);

function maskS2Sr(image) {
  var scl = image.select('SCL');
  var clear = scl.neq(3)
    .and(scl.neq(8))
    .and(scl.neq(9))
    .and(scl.neq(10))
    .and(scl.neq(11));
  return image.updateMask(clear)
    .select(['B4', 'B8'], ['red', 'nir'])
    .multiply(0.0001);
}

function annualNdvi(year) {
  year = Number(year);
  var start = ee.Date.fromYMD(year, 4, 1);
  var end = ee.Date.fromYMD(year, 11, 1);
  var collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate(start, end)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 60))
    .map(maskS2Sr)
    .map(function(image) {
      return image.normalizedDifference(['nir', 'red']).rename('NDVI');
    });
  return collection.max().clip(roi);
}

var ndviVis = {
  min: 0,
  max: 0.9,
  palette: ['8c510a', 'f6e8c3', 'c7eae5', '5ab4ac', '01665e']
};
```

```
// ----- 2. 创建独立地图和侧栏 -----
var appMap = ui.Map();
appMap.setCenter(116.4, 40.0, 9);

var title = ui.Label('北京示例区年度最大 NDVI', {
  fontSize: '22px',
  fontWeight: 'bold',
  color: '#173A5E',
  margin: '0 0 12px 0'
});

var intro = ui.Label(
  '选择年份后，应用使用 Sentinel-2 SR Harmonized，完成 SCL 去云并显示 4—10 月最大 NDVI。',
  {whiteSpace: 'pre-wrap', margin: '0 0 12px 0'}
);

var status = ui.Label('准备就绪', {color: '#5D6B78'});

var yearSelect = ui.Select({
  items: ['2022', '2023', '2024', '2025'],
  value: '2025',
  placeholder: '选择年份'
});

function drawLegend() {
  var legend = ui.Panel({style: {margin: '16px 0 0 0'}});
  legend.add(ui.Label('NDVI: 0 → 0.9', {fontWeight: 'bold'}));
  legend.add(ui.Thumbnail({
    image: ee.Image.pixelLonLat().select('longitude'),
    params: {
      bbox: [0, 0, 1, 0.1],
      dimensions: '240x20',
      format: 'png',
      min: 0,
      max: 1,
      palette: ndviVis.palette
    },
    style: {stretch: 'horizontal', margin: '6px 0'}
  }));
  return legend;
}

function updateMap(year) {
  status.setValue('正在准备 ' + year + ' 年图层...');
  var image = annualNdvi(year);

  // reset() 保证每次选择年份只保留一个主图层。
  appMap.layers().reset([
    ui.Map.Layer(image, ndviVis, year + ' 年最大 NDVI')
  ]);
  appMap.addLayer(ee.Image().paint(roi, 1, 2), {palette: ['00FFFF'], '研究区边界'});
  status.setValue('当前年份: ' + year + '；可点击地图用 Inspector 查看像元。');
}

yearSelect.onChange(updateMap);

var sidePanel = ui.Panel({
  widgets: [
    title,
    intro,
    ui.Label('年份', {fontWeight: 'bold'}),
    yearSelect,
    status,
    drawLegend(),
    ui.Label(
      '注意：本应用用于教学。正式科研应记录数据访问日期、质量掩膜、时间窗口和验证结果。',
      {whiteSpace: 'pre-wrap', margin: '18px 0 0 0', color: '#8A4B08'}
    )
  ]
});
```

```

    )
  ],
  style: {width: '330px', padding: '18px'}
});

// ----- 3. 重建应用界面 -----
ui.root.clear();
ui.root.setLayout(ui.Panel.Layout.flow('horizontal'));
ui.root.add(sidePanel);
ui.root.add(appMap);

// 首次打开应用时主动绘制默认年份。
updateMap('2025');

// 发布前在 Apps 面板创建 App，选择 Cloud 项目并设置访问方式。
// 如果改用个人 Asset，必须同时给 App 或目标用户相应 Asset 读取权限。

```

结果说明：在 Code Editor 运行后，界面会被重建为左侧说明/年份选择和右侧独立地图。切换年份应替换主 NDVI 图层并更新状态。发布前模板使用公开 Sentinel-2 和内置矩形，因此没有个人 Asset 依赖。公开项目至少包含“可访问代码、可访问数据或替代数据、运行说明、方法限制、许可和版本”。

3.4.5 常见报错解决

- App 作者可见、访客空白：个人 Asset 未共享给 App/公众，或 App Cloud 项目配置错误。
- App 加载很慢：首屏计算过重；缩小默认 ROI、预计算资产、提高 scale、延迟到用户点击后运行。
- GitHub 代码运行失败：README 漏写 Asset、Cloud 项目或数据迁移信息。
- Get Link 打开后版本不同：明确快照链接与仓库最新版本的区别，并打版本标签。
- 公开后担心代码泄露：App 客户端源码不可作为秘密容器；立即移除凭据并轮换已暴露密钥。

3.5 六遍读懂一段 GEE 代码

3.5.1 阅读目标

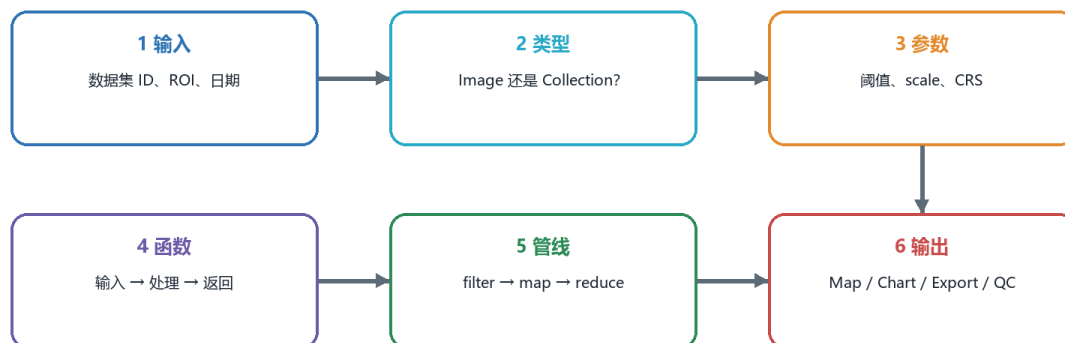
阅读代码不是逐字翻译 API，而是复述数据流：“从什么数据，在什么空间和时间，经过什么质量控制与统计，得到什么地图、表格和导出。”

3.5.2 六遍阅读法

1. 输入：圈出数据集 ID、ROI、开始/结束日期和个人 Asset。
2. 类型：在每个主要变量旁标注 Geometry、FeatureCollection、ImageCollection、Image、Number、Dictionary 或 UI 对象。
3. 参数：记录云量、阈值、scale、CRS、maxPixels、palette 与导出范围；区分“科学参数”和“仅显示参数”。
4. 函数：对每个 function 写下输入类型、返回类型、是否保留 system:time_start、是否改变掩膜或单位。
5. 管线：把链式调用翻译为动词：先 filterBounds、再 filterDate、再 map、再 median/max、再 clip。
6. 输出与验收：找出 print、Map.addLayer、ui.Chart、Export；询问每个结果由什么证据证明正确。

六遍读懂一段 GEE 代码

每一遍只回答一个问题：先理解对象和数据流，再纠结单个 API



读完后应能用一句话复述：从什么数据，在什么范围和时间，经过什么处理，得到什么结果。

图 7 六遍 GEE 代码阅读法：输入、对象类型、参数、函数、处理管线和输出/验证。

3.5.3 带类型标注的阅读练习

```
// [输入] roi: Geometry; images: ImageCollection。
var roi = ee.Geometry.Rectangle([116.2, 39.8, 116.5, 40.1]);
var images = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi) // ImageCollection → ImageCollection
  .filterDate('2025-04-01', '2025-11-01')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 30));

// [函数] 输入一景 Image，输出增加 NDVI 波段的 Image。
function addNdvi(image) {
  var scaled = image.select(['B4', 'B8']).multiply(0.0001);
  var ndvi = scaled.normalizedDifference(['B8', 'B4']).rename('NDVI');
  return image.addBands(ndvi);
}

// [管线] map 后仍是 ImageCollection；max 后才是单张 Image。
var annualMax = images.map(addNdvi).select('NDVI').max().clip(roi);

// [输出] 显示不改变 annualMax 的数值。
print('数量', images.size());
print('输出波段', annualMax.bandNames());
Map.addLayer(annualMax, {min: 0, max: 0.9, palette: ['white', 'green']}, 'NDVI');
```

阅读后应能发现：示例只做了整景云量筛选，没有像元级去云；函数里的 `scaled` 只有 B4/B8，而返回的是原始 `image.addBands(ndvi)`；NDVI 已正确由缩放后波段计算，但原始波段仍保持整数；若改成时序图，需要保留/确认时间属性。这些判断比背诵 `normalizedDifference` 更重要。

GEE 阅读的核心是对象类型和数据流。每次方法调用都问“谁调用、输入什么、返回什么、单位/掩膜/属性是否改变”。

3.5.4 常见报错解决

- 链式调用看不懂：拆成多个有意义的中间变量，并逐个 `print()`。
- 不知道返回类型：在 Docs 搜方法，看 Returns；不要凭变量名猜。
- `map` 函数结果奇怪：检查函数是否 `return`，返回类型是否统一，时间属性是否保留。
- 能运行但说不出意义：写一行伪代码摘要，并列出三个科学假设。

3.6 GPT × GEE 的 Vibe Coding：从提示词到科学验收

3.6.1 正确定义 Vibe Coding

这里的 Vibe Coding 不是“描述一句话，让 GPT 生成几百行后直接用于论文”，而是人机协作的快速原型：人定义研究问题、数据、尺度、约束和验收证据；GPT 生成或修改最小代码；人在 GEE 中运行并完成 API、数值和科学验证。OpenAI 官方模型指导建议用结果导向的提示词说明目标、上下文、约束、所需证据、成功标准和输出格式；本专题把这一原则具体化为 GEE 任务契约。

GPT × GEE：可验证的 Vibe Coding 闭环

模型负责加速表达与重构；研究者负责数据、尺度、假设和证据

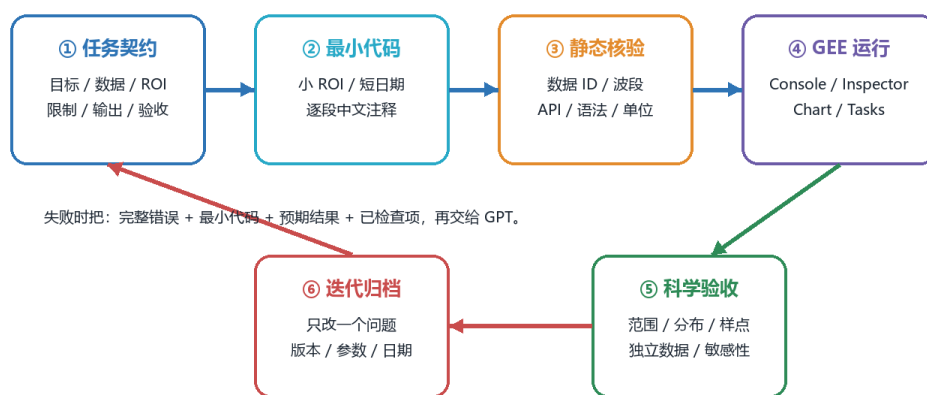


图 8 GPT × GEE 可验证闭环。失败时不是让模型“再试一次”，而是回传完整错误、最小代码、预期结果和已检查证据。

3.6.2 提示词的八项任务契约

1. 角色与环境：明确 Google Earth Engine JavaScript Code Editor，不要混入 Python/geemap 语法。
2. 科学目标：用一句可验收的话描述，例如“计算 2018—2025 每年 4—10 月森林像元最大 NDVI”。
3. 数据：给出准确 Data Catalog ID、波段、比例因子和 QA/去云要求；没有把握时要求提供官方链接并标记不确定。
4. ROI：绘制几何、公开表或个人 Asset ID；说明是否要保留分区属性。

5. 时间：写完整起止日期，并注意 filterDate(start, end) 的结束日期是排他的。
6. 处理：按顺序列出筛选、掩膜、缩放、指数、合成、统计和缺测处理。
7. 输出：地图图层、Console、图表、Drive/Asset 导出，明确字段、scale、CRS 与单位。
8. 验收：集合非空、波段存在、理论范围、分位数、随机像元、外部数据、敏感性与版本记录。

3.6.3 一个高质量生成提示词

```

你是一名熟悉 GEE JavaScript Code Editor 的遥感工程师。
目标：在北京示例矩形 [116.05,39.70,116.75,40.25] 内，计算 2024 年
4 月 1 日至 11 月 1 日 Sentinel-2 年内最大 NDVI。

数据必须使用 COPERNICUS/S2_SR_HARMONIZED；用 SCL 去除 3、8、9、10、11；
B4/B8 乘 0.0001。先给小 ROI 的最小可运行版本，使用 var 和普通 function，
逐行中文注释。输出固定 min=0,max=0.9 的 NDVI 图层、影像数量、bandNames、
min/max、分位数、直方图，以及一个 Export.image.toDrive 任务（scale=10）。

不要编造数据集、波段或 API；不确定处标记并给 Earth Engine 官方文档链接。
验收：size>0，输出仅含 NDVI，值在理论 -1—1 内，随机点击 5 个像元，
并说明 3 个不能仅靠地图颜色验证的科学风险。

```

不好的提示词通常只有“帮我计算 NDVI 并导出”，缺少数据层级、云掩膜、ROI、时间、缩放、输出尺度和验收，因此模型只能猜用户的真实意图。

3.6.4 调试提示词：给证据，不给情绪

把以下四样交给 GPT：完整错误文本、最小复现代码、预期结果、已检查项。不要只说“代码报错了”。要求模型先归类为语法、对象类型、空集合、权限、资源限制或科学逻辑，再做最小修改，并返回运行后应检查的证据。

```

错误：Image.select: Band pattern 'B8' did not match any bands.
最小代码：[粘贴相关 20—60 行]
预期：得到 NDVI Image。
已检查：collection.size()>0；请让我打印 first().bandNames()，不要猜波段。
只修改根因：解释修改前后类型，并给 3 项运行验收。

```

3.6.5 三层真实性核验

层级	要回答的问题	证据
语法/API	是否能解析，方法和参数是否存在	Code Editor 错误、Docs、官方 API Reference
数据/数值	ID、波段、比例因子、掩膜、范围、单位是否正确	Data Catalog、size、bandNames、projection、min/max、分位数
科学结论	指标是否代表目标过程，阈值和趋势是否稳健	样地/高分影像、跨传感器、一致时间窗、敏感性、不确定性

GPT 可以生成语法正确但科学错误的代码，例如把整景云量阈值误当像元级去云、把 NDVI 改善直接解释为造林、混用不同时间窗口、在不同投影尺度下比较面积。通过 Run 只是第一层通过。

3.6.6 完整最小验收脚本

将 GPT 生成的候选结果替换到 candidate，再逐项运行。以下示例本身也可直接运行。

```
// =====
// 08 GPT 生成 GEE 代码后的最小验收脚本
// 目的：不靠“地图看起来正常”，而用数量、波段、范围和图表留证。
// =====

var roi = ee.Geometry.Rectangle([116.20, 39.80, 116.55, 40.10]);
var start = '2024-06-01';
var end = '2024-09-01';

function maskS2Sr(image) {
  var scl = image.select('SCL');
  var clear = scl.neq(3)
    .and(scl.neq(8))
    .and(scl.neq(9))
    .and(scl.neq(10))
    .and(scl.neq(11));
  return image.updateMask(clear)
    .select(['B4', 'B8'], ['red', 'nir'])
    .multiply(0.0001)
    .copyProperties(image, ['system:time_start']);
}

var source = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(roi)
  .filterDate(start, end)
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 60))
  .map(maskS2Sr);

var ndviCollection = source.map(function(image) {
  return image.normalizedDifference(['nir', 'red'])
    .rename('NDVI')
    .copyProperties(image, ['system:time_start']);
});
var candidate = ndviCollection.median().rename('NDVI').clip(roi);

// ----- 1. 输入验收 -----
print('QC-01 日期范围', start, end);
print('QC-02 ROI 面积 km²', roi.area(1).divide(1e6));
print('QC-03 影像数量（必须 > 0）', source.size());
print('QC-04 首景波段（应为 red, nir）', ee.Image(source.first()).bandNames());

// ----- 2. 输出结构与数值验收 -----
print('QC-05 候选结果波段（应为 NDVI）', candidate.bandNames());
print('QC-06 候选结果投影', candidate.projection());

var minMax = candidate.reduceRegion({
  reducer: ee.Reducer.minMax(),
  geometry: roi,
  scale: 30,
  maxPixels: 1e8,
  tileScale: 2
});
print('QC-07 NDVI 最小/最大值（理论范围 -1 到 1）', minMax);

var percentiles = candidate.reduceRegion({
  reducer: ee.Reducer.percentile([2, 25, 50, 75, 98]),
  geometry: roi,
  scale: 30,
  maxPixels: 1e8,
  tileScale: 2
});
print('QC-08 NDVI 分位数（检查异常分布）', percentiles);

// ----- 3. 空间与时序验收 -----
Map.centerObject(roi, 10);
Map.addLayer(candidate, {
  min: 0,
  max: 0.9,
```

```

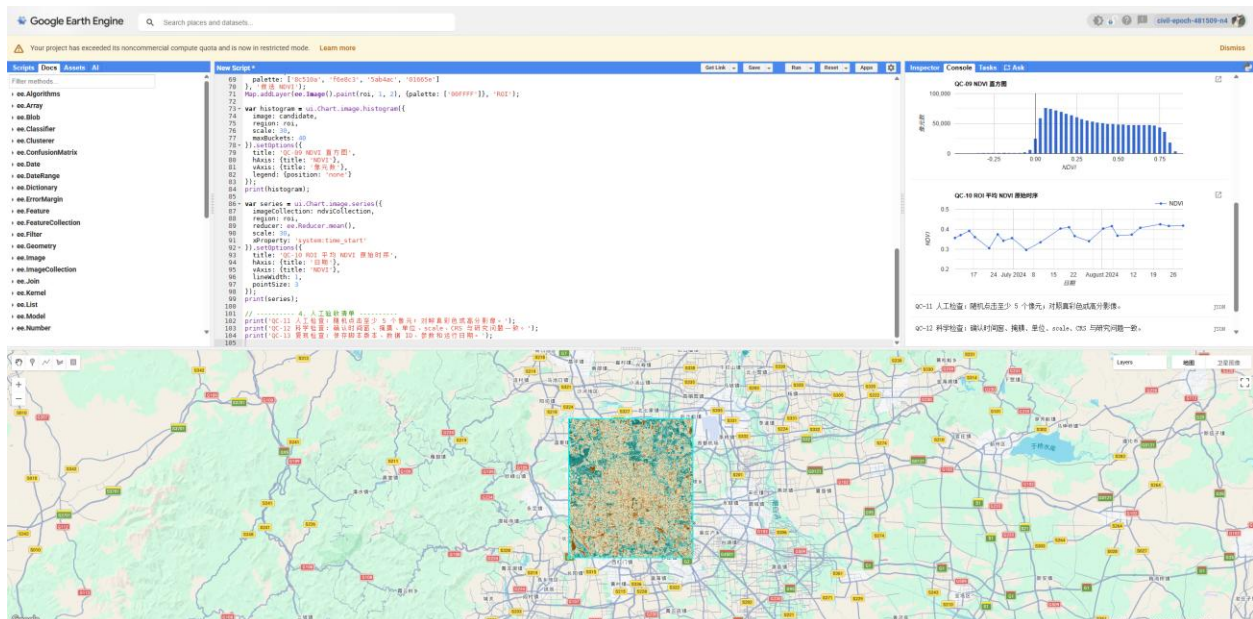
palette: ['8c510a', 'f6e8c3', '5ab4ac', '01665e']
}, '候选 NDVI');
Map.addLayer(ee.Image().paint(roi, 1, 2), {palette: ['00FFFF']}, 'ROI');

var histogram = ui.Chart.image.histogram({
  image: candidate,
  region: roi,
  scale: 30,
  maxBuckets: 40
}).setOptions({
  title: 'QC-09 NDVI 直方图',
  hAxis: {title: 'NDVI'},
  vAxis: {title: '像元数'},
  legend: {position: 'none'}
});
print(histogram);

var series = ui.Chart.image.series({
  imageCollection: ndviCollection,
  region: roi,
  reducer: ee.Reducer.mean(),
  scale: 30,
  xProperty: 'system:time_start'
}).setOptions({
  title: 'QC-10 ROI 平均 NDVI 原始时序',
  hAxis: {title: '日期'},
  vAxis: {title: 'NDVI'},
  lineWidth: 1,
  pointSize: 3
});
print(series);

// ----- 4. 人工验收清单 -----
print('QC-11 人工检查：随机点击至少 5 个像元；对照真彩色或高分影像。');
print('QC-12 科学检查：确认时间窗、掩膜、单位、scale、CRS 与研究问题一致。');
print('QC-13 复现检查：保存脚本版本、数据 ID、参数和运行日期。');

```



3.6.7 结果正确性检查表

- 输入：Data Catalog 页面仍存在；时间和 ROI 交集非空；size()>0。
- 结构：bandNames()、属性名、数据类型、投影与名义分辨率符合预期。

- 单位：反射率比例因子、温度偏移、面积单位和趋势单位写入注释。
- 范围：NDVI 理论 -1—1；打印 min/max 与 2/98 分位数，不用显示范围替代统计。
- 空间：随机点击至少 5 个典型地物；边界、掩膜、拼接缝和异常条带可解释。
- 时间：图表没有 1970 年；每年观测数、缺测与部分年度标记清楚。
- 统计：reducer、scale、CRS、mask、分母和 maxPixels/tileScale 记录一致。
- 独立验证：样地、高分影像、另一数据源或公开产品至少一种；报告不一致。
- 敏感性：改变云掩膜、合成方法、趋势阈值或 scale，结论方向是否稳定。
- 复现：保存原始提示词、GPT 输出、人工修改、脚本版本、运行日期和官方来源。

3.6.8 数据安全与作者责任

不要把 Google 密码、Cloud 凭据、API Key、服务账号密钥、未公开样地、个人信息或敏感物种精确位置发送给 GPT。可用字段结构、匿名样例和缩略 ROI 复现问题。AI 生成内容必须由作者承担审查、许可、引用和科研结论责任。完整可复制提示词见代码包 GPT_GEE 提示词模板.md。

OpenAI 官方依据：[Model guidance—Prompting best practices](#)。

Vibe Coding 把 GPT 当作“代码合作者”和“审稿人”，不要当作数据目录或科学真理。提示词要具体，生成要最小，验证要分层，修改要可追踪。

3.6.9 常见报错解决

- GPT 编造数据 ID/波段：从官方 Data Catalog 复制，不用社区博客二次转述。
- 代码混入 Python：提示明确 Code Editor JavaScript；要求使用 var、function 和 ee.*。
- 修一个错又改坏全篇：要求最小差异，只修改根因；保存上一版。
- 代码运行却结果可疑：执行验收脚本，打印范围、分位数、原始时序并做独立验证。
- 对话太长、要求互相冲突：重新发送一份精简任务契约，删除重复要求，固定成功标准。

4 完整科研实战：2018—2026 森林植被动态监测

4.1 项目设计、研究问题与数据选择

4.1.1 项目主题

研究主题：基于 GEE 的长时间序列森林植被动态监测与植被变化分析。

示范研究区位于西双版纳附近，采用可直接运行的矩形范围 [100.55, 21.72, 101.10, 22.22]。为演示分区统计，将 ROI 均分为西北、东北、西南、东南四区。正式论文应替换为有来源和投影说明的行政边界、保护区边界或样地范围。

4.1.2 研究问题

1. 2018—2026 年森林年度最大 NDVI 的空间格局如何？
2. 哪些像元呈持续改善、稳定或退化？
3. 四个分区的变化面积和高植被覆盖占比是否不同？
4. 结果对数据源、时间窗、森林掩膜和阈值是否敏感？

4.1.3 数据选择

默认使用 COPENICUS/S2_SR_HARMONIZED，理由是 10 m 红光/近红外波段、2018 年以来覆盖稳定，并对 2022 年处理基线变化进行了 DN harmonization。脚本可把 DATA_SOURCE 改为 LANDSAT，自动合并 Landsat 8/9 Collection 2 Level 2，统一为 blue/green/red/nir/swir1 波段。

森林样本框采用 ESA/WorldCover/v200 的 2021 树木覆盖类（Map=10）。它保证分析聚焦于基准年树木覆盖区，但不能代表每一年动态森林边界。若论文研究森林面积增减，应另外构建年度森林分类或使用独立森林变化产品。

4.1.4 2026 年可比性设计

2026 年尚未结束。若用 2018—2025 完整自然年与 2026 部分年度比较，最大值合成的观测机会不一致。项目默认对每一年都使用 1 月 1 日—7 月 29 日的相同窗口，因此得到“同窗年度最大 NDVI”。年末更新时，可把结束月日改为 12 月 31 日并重新运行全部年份。

长时序分析的第一原则是可比性：相同传感器体系、反射率层级、质量控制、季节窗口、森林定义和空间尺度。

4.1.5 常见报错解决

- 把 2026 结果称为完整年度：在图题、表头和方法中明确“截至 7 月 29 日同窗”。
- S2 与 Landsat 数值直接拼接：空间分辨率、光谱响应和观测频率不同；本课程用于敏感性比较，不默认混成单一序列。
- 用 2021 森林掩膜声称年度森林面积：静态掩膜只定义分析样本框。

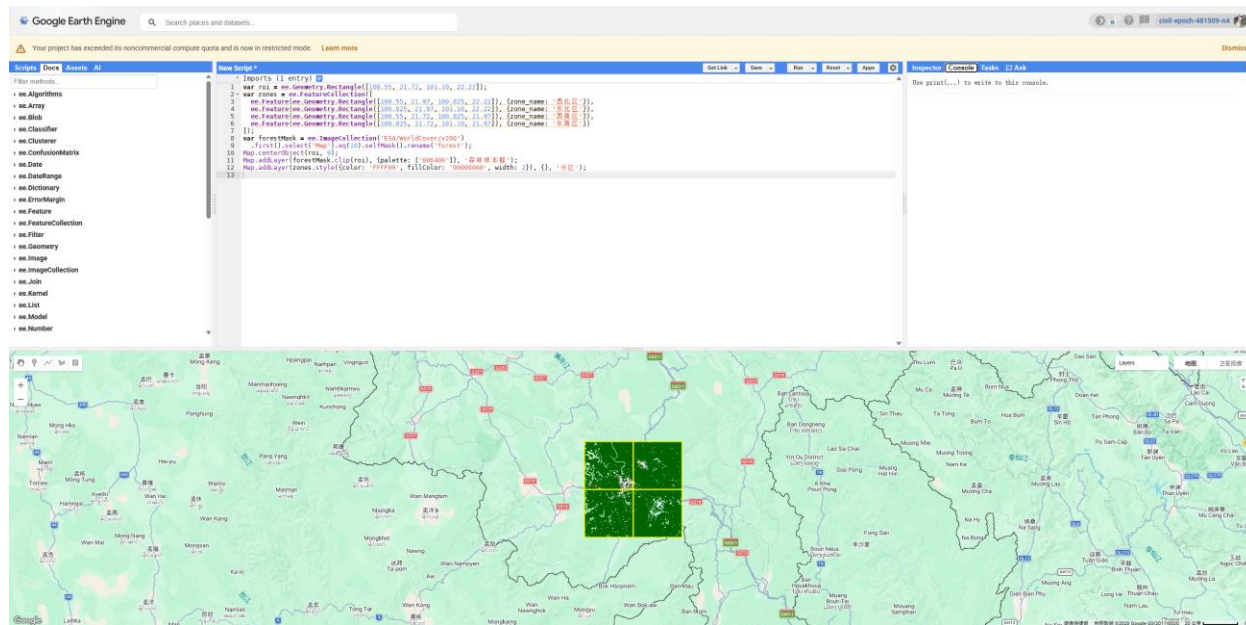
4.2 研究区、分区与森林样本框

4.2.1 知识点讲解

研究区影响计算规模和推论范围；分区是统计单元；森林掩膜是样本选择条件。三者不要混为一谈。正式研究应记录边界来源、日期、比例尺/精度和授权。

4.2.2 可运行代码

```
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var zones = ee.FeatureCollection([
  ee.Feature(ee.Geometry.Rectangle([100.55, 21.97, 100.825, 22.22]), {zone_name: '西北区'}),
  ee.Feature(ee.Geometry.Rectangle([100.825, 21.97, 101.10, 22.22]), {zone_name: '东北区'}),
  ee.Feature(ee.Geometry.Rectangle([100.55, 21.72, 100.825, 21.97]), {zone_name: '西南区'}),
  ee.Feature(ee.Geometry.Rectangle([100.825, 21.72, 101.10, 21.97]), {zone_name: '东南区'})
]);
var forestMask = ee.ImageCollection('ESA/WorldCover/v200')
  .first().select('Map').eq(10).selfMask().rename('forest');
Map.centerObject(roi, 9);
Map.addLayer(forestMask.clip(roi), {palette: ['006400'], '森林样本框'});
Map.addLayer(zones.style({color: 'FFFF00', fillColor: '00000000', width: 2}), {}, '分区');
```



结果说明：地图显示 2021 树木覆盖像元和四个分区。用 Inspector 检查林区、农地和城镇附近，理解土地覆盖产品不是地面真值。静态森林样本框适合研究“基准森林范围内植被状态变化”，不等同于年度森林面积制图。

4.2.3 常见报错解决

- WorldCover 写成 ee.Image('ESA/WorldCover/v200/2021') 后困惑：目录对象是 ImageCollection；课程统一使用 .first()，更贴合目录定义。
- 分区重叠：检查经纬度边界，面积占比统计前确认几何关系。
- ROI 太大：先缩小验证全流程，再分块扩展。

4.3 Sentinel-2/Landsat 批量获取与预处理

4.3.1 Sentinel-2 路线

SCL 去除无数据（0）、饱和/坏像元（1）、云影（3）、中概率云（8）、高概率云（9）、卷云（10）和雪冰（11）；选 B2/B3/B4/B8/B11 并统一命名；乘 0.0001 转为反射率。

4.3.2 Landsat 路线

合并 LC08/LC09 C02 T1 L2；QA_PIXEL 低 6 位剔除填充值、膨胀云、卷云、云、云影和雪；QA_RADSAT 剔除饱和；SR 波段乘 0.0000275 再加 -0.2。两条路线输出相同语义波段，后续 NDVI 代码无需修改。

4.3.3 可运行核心代码

```
function prepareSentinel2(image) {
  var scl = image.select('SCL');
  var mask = scl.neq(0).and(scl.neq(1)).and(scl.neq(3))
    .and(scl.neq(8)).and(scl.neq(9))
    .and(scl.neq(10)).and(scl.neq(11));
  return image.updateMask(mask)
    .select(['B2', 'B3', 'B4', 'B8', 'B11'], ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0001)
    .copyProperties(image, ['system:time_start']);
}

function prepareLandsat(image) {
  var mask = image.select('QA_PIXEL').bitwiseAnd(63).eq(0)
    .and(image.select('QA_RADSAT').eq(0));
  return image.updateMask(mask)
    .select(['SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6'],
      ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0000275).add(-0.2)
    .copyProperties(image, ['system:time_start']);
}
```

4.3.4 结果说明

预处理后的影像应包含统一命名的五个反射率波段，典型地表反射率大体在 0—1，允许少量负值或大于 1 的有效物理/处理异常值。不要用 clamp(0,1) 静默修改科研数据，除非有明确方法依据。统一波段语义让算法可复用；但统一命名并未消除传感器差异。跨传感器结论仍需敏感性分析或校准。

4.3.5 常见报错解决

- Landsat 颜色异常：忘记缩放和偏移，或把 L1 波段公式用于 L2。
- S2 边缘噪声：可进一步结合 B8A/B9 边缘掩膜或 Cloud Score+；课程保持入门代码简洁。
- 云量阈值后无影像：整景云量只是初筛；适度放宽到 50—60，再依赖像元掩膜。

4.4 年度最大 NDVI 合成

4.4.1 原理

对每景无云反射率计算 NDVI，再在每个统一年度窗口逐像元取最大值。最大值合成有利于弱化残云和短期低值，但可能偏向异常高值，并且每个像元的观测日期不同。论文应把它描述为“maximum-value composite”，不能称为某一天影像。

4.4.2 可运行核心代码

```
function addNdvi(image) {
  var ndvi = image.normalizedDifference(['nir', 'red']).rename('NDVI');
  return image.addBands(ndvi).copyProperties(image, ['system:time_start']);
}
var withNdvi = optical.map(addNdvi);
var years = ee.List.sequence(2018, 2026);
var annualNdvi = ee.ImageCollection.fromImages(years.map(function(year) {
  year = ee.Number(year);
  var start = ee.Date.fromYMD(year, 1, 1);
  var end = ee.Date.fromYMD(year, 7, 29).advance(1, 'day');
  return withNdvi.filterDate(start, end).select('NDVI').max()
    .rename('NDVI').updateMask(forestMask).clip(roi)
    .set('year', year)
    .set('system:time_start', start.millis());
})));
```

结果说明：集合应有 9 景年度影像。分别打开 2018 和 2026 图层，检查空间覆盖、残云、接边与极端值；用区域时序图确认没有突兀的全区跳变。年度合成是观测策略，不只是代码操作。固定窗口、质量掩膜和有效观测数决定趋势可信度。

4.4.3 常见报错解决

- 某年全空：检查数据源起始时间、ROI 覆盖、filterDate 结束日期和去云阈值。
- 最大 NDVI 接近 1 且呈斑点：可能薄云边缘或异常值；改用 90/95 百分位合成做敏感性比较。
- 2026 值系统偏低：确认所有年份使用相同截止日。

4.5 线性趋势、Sen's slope 与 Kendall tau

4.5.1 指标解释

- 线性斜率：最小二乘拟合的 NDVI 年变化率，单位 NDVI/年，易解释但受异常值影响。
- Sen's slope：所有时间点对的斜率中位数，对异常值更稳健。
- Kendall tau：秩相关的单调趋势强度，范围 -1—1；正值表示总体上升，负值表示总体下降。

项目要求至少 7 个有效年度。分类同时使用 Sen's slope 幅度和 Kendall tau 强度：改善为 slope > 0.005 且 tau > 0.40；退化为 slope < -0.005 且 tau < -0.40；其余为稳定。阈值是教学默认值，不是普适生态阈值。正式论文应进行阈值敏感性分析，并视研究设计增加 Mann-Kendall 显著性、空间自相关和多重比较处理。

4.5.2 可运行核心代码

```
var validYears = annualNdvi.count().gte(7);
var withTime = annualNdvi.map(function(image) {
  var t = image.select('NDVI').multiply(0)
    .add(ee.Number(image.get('year')).subtract(2018)).rename('t').float();
  return t.addBands(image.select('NDVI').float());
});
```

```
});
var linearSlope = withTime.select(['t', 'NDVI'])
  .reduce(ee.Reducer.linearFit()).select('scale').updateMask(validYears);
var senSlope = withTime.select(['t', 'NDVI'])
  .reduce(ee.Reducer.sensSlope()).select('slope').updateMask(validYears);
var tau = withTime.select(['t', 'NDVI'])
  .reduce(ee.Reducer.kendallsCorrelation(2)).rename('kendall_tau').updateMask(validYears);
```

结果说明：斜率为 0.01 表示拟合意义下每年 NDVI 增加约 0.01。tau 接近 1 表示序列较一致地上升；斜率高但 tau 低，可能由少数年份驱动。“变化幅度”和“变化一致性”是不同维度。双指标比简单首尾差值更稳健，但仍不能自动证明因果关系。

4.5.3 常见报错解决

- slope 单位看不懂：时间波段用“距 2018 的年数”，所以单位是 NDVI/年。
- reducer 输入顺序错误：必须先 x 时间、后 y NDVI。
- 趋势图大面积空白：有效年份不足或森林掩膜太严；显示 valid_years 检查。

4.6 变化提取与各区域植被占比

4.6.1 变化分类

```
var improve = senSlope.gt(0.005).and(tau.gt(0.40));
var degrade = senSlope.lt(-0.005).and(tau.lt(-0.40));
var changeClass = senSlope.multiply(0).toInt8()
  .where(improve, 1)
  .where(degrade, -1)
  .rename('change_class')
  .updateMask(validYears);
```

编码为 -1 退化、0 稳定、1 改善。保持整数编码可用于 GIS 制图和面积统计。

4.6.2 面积与占比原理

使用 ee.Image.pixelArea() 获得平方米，再除以 1e6 转平方千米。面积统计应使用明确 scale。课程在 100 m 做分区统计以控制计算量；像元级结果仍可按原生尺度导出。高植被覆盖占比定义为：某分区森林有效像元中，年度最大 NDVI ≥ 0.60 的面积比例。它是课程指标，不等同于冠层覆盖度。阈值应按区域生态特征、样地和影像分布校准。

4.6.3 常见报错解决

- 三类百分比和不等 100：检查掩膜是否一致、统计 scale 是否一致和四舍五入误差。
- 面积随地图缩放变化：分析时没有在 reduceRegion 中指定 scale。
- 分区返回 null：该分区没有有效像元；不要直接 ee.Number(null)，正式脚本应先检查样本。

4.7 完整项目代码与复现实验

4.7.1 完整可直接运行脚本

```
// =====
// 完整实战项目：基于 GEE 的长时间序列森林植被动态监测与变化分析
// 研究区：西双版纳教学示范区（矩形 + 四个教学分区）
// 时段：2018—2026；为保证年际可比，默认每年使用 1 月 1 日—7 月 29 日
// 数据：默认 Sentinel-2 SR Harmonized，可一键切换 Landsat 8/9 C2 L2
```

```

// 输出：年度最大 NDVI、线性斜率、Sen's slope、Kendall tau、变化分类、统计表
// =====

// -----
// 0. 用户参数：通常只需修改这里
// -----
var DATA_SOURCE = 'S2';    // 可选：'S2' 或 'LANDSAT'
var START_YEAR = 2018;
var END_YEAR = 2026;
var WINDOW_END_MONTH = 7;   // 每年统一分析窗口的结束月
var WINDOW_END_DAY = 29;    // 每年统一分析窗口的结束日
var MAX_SCENE_CLOUD = 40;   // 整景云量初筛阈值（%）
var MIN_VALID_YEARS = 7;    // 至少有 7 个有效年度才进行趋势分类
var SLOPE_THRESHOLD = 0.005; // NDVI/年；变化幅度阈值
var TAU_THRESHOLD = 0.40;   // Kendall tau；单调趋势强度阈值
var VEGETATION_NDVI = 0.60; // “高植被覆盖”教学阈值
var EXPORT_FOLDER = 'GEE_Forest_NDVI_Project';

// Sentinel-2 原生 NDVI 分辨率为 10 m；Landsat 为 30 m。
var NATIVE_SCALE = DATA_SOURCE === 'S2' ? 10 : 30;
// 分区统计用 100 m，显著降低计算量；论文中可按研究需求改为原生尺度。
var STAT_SCALE = 100;

// -----
// 1. 研究区与四个教学分区
// -----
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var midLon = 100.825;
var midLat = 21.97;

var zones = ee.FeatureCollection([
  ee.Feature(
    ee.Geometry.Rectangle([100.55, midLat, midLon, 22.22]),
    {zone_id: 1, zone_name: '西北区'}
  ),
  ee.Feature(
    ee.Geometry.Rectangle([midLon, midLat, 101.10, 22.22]),
    {zone_id: 2, zone_name: '东北区'}
  ),
  ee.Feature(
    ee.Geometry.Rectangle([100.55, 21.72, midLon, midLat]),
    {zone_id: 3, zone_name: '西南区'}
  ),
  ee.Feature(
    ee.Geometry.Rectangle([midLon, 21.72, 101.10, midLat]),
    {zone_id: 4, zone_name: '东南区'}
  )
]);

Map.centerObject(roi, 9);
Map.addLayer(ee.Image().byte().paint(roi, 1, 2), {palette: ['FFFFFF']}, '研究区边界');
Map.addLayer(
  ee.Image().byte().paint(zones, 'zone_id', 2),
  {min: 1, max: 4, palette: ['00FFFF', 'FF00FF', 'FFFF00', '00FF00']},
  '四个教学分区'
);

// -----
// 2. 森林样本框：ESA WorldCover 2021 树木覆盖类（Map = 10）
// -----
var worldCover = ee.ImageCollection('ESA/WorldCover/v200').first().select('Map');
var forestMask = worldCover.eq(10).selfMask().rename('forest');
Map.addLayer(forestMask.clip(roi), {palette: ['006400']}, '2021 树木覆盖范围', false);

// -----
// 3A. Sentinel-2 预处理：SCL 去云 + 反射率缩放 + 波段统一命名
// -----

```

```

function prepareSentinel2(image) {
  var scl = image.select('SCL');
  var clearMask = scl.neq(0) // 无数据
    .and(scl.neq(1)) // 饱和或坏像元
    .and(scl.neq(3)) // 云影
    .and(scl.neq(8)) // 中概率云
    .and(scl.neq(9)) // 高概率云
    .and(scl.neq(10)) // 卷云
    .and(scl.neq(11)); // 雪/冰

  return image
    .updateMask(clearMask)
    .select(['B2', 'B3', 'B4', 'B8', 'B11'],
      ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0001)
    .copyProperties(image, ['system:time_start']);
}

// -----
// 3B. Landsat 8/9 预处理：QA_PIXEL 去云 + 饱和像元剔除 + 缩放
// -----
function prepareLandsat(image) {
  var qa = image.select('QA_PIXEL');
  // 低 6 位依次包含填充值、膨胀云、卷云、云、云影、雪。
  var clearMask = qa.bitwiseAnd(63).eq(0);
  var saturationMask = image.select('QA_RADSAT').eq(0);

  return image
    .updateMask(clearMask)
    .updateMask(saturationMask)
    .select(['SR_B2', 'SR_B3', 'SR_B4', 'SR_B5', 'SR_B6'],
      ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0000275)
    .add(-0.2)
    .copyProperties(image, ['system:time_start']);
}

// -----
// 4. 批量获取光学影像：返回统一的 blue/green/red/nir/swir1 波段
// -----
var globalStart = ee.Date.fromYMD(START_YEAR, 1, 1);
var globalEnd = ee.Date.fromYMD(
  END_YEAR,
  WINDOW_END_MONTH,
  WINDOW_END_DAY
).advance(1, 'day');

function getOpticalCollection() {
  if (DATA_SOURCE === 'LANDSAT') {
    var landsat8 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2');
    var landsat9 = ee.ImageCollection('LANDSAT/LC09/C02/T1_L2');
    return landsat8.merge(landsat9)
      .filterBounds(roi)
      .filterDate(globalStart, globalEnd)
      .filter(ee.Filter.lt('CLOUD_COVER', MAX_SCENE_CLOUD))
      .map(prepareLandsat);
  }

  return ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate(globalStart, globalEnd)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', MAX_SCENE_CLOUD))
    .map(prepareSentinel2);
}

var optical = getOpticalCollection();
print('当前数据源', DATA_SOURCE);
print('预处理后的影像总数', optical.size());

```

```

// -----
// 5. 为每景影像计算 NDVI
// -----
function addNdvi(image) {
  var ndvi = image.normalizedDifference(['nir', 'red']).rename('NDVI');
  return image.addBands(ndvi)
    .copyProperties(image, ['system:time_start']);
}

var withNdvi = optical.map(addNdvi);

// -----
// 6. 生成 2018—2026 同窗年度最大 NDVI 合成
// -----
var years = ee.List.sequence(START_YEAR, END_YEAR);

var annualNdvi = ee.ImageCollection.fromImages(
  years.map(function(year) {
    year = ee.Number(year);
    var start = ee.Date.fromYMD(year, 1, 1);
    // filterDate 的结束日期不包含当天，所以向后推进 1 天。
    var end = ee.Date.fromYMD(
      year,
      WINDOW_END_MONTH,
      WINDOW_END_DAY
    ).advance(1, 'day');

    var annualMax = withNdvi
      .filterDate(start, end)
      .select('NDVI')
      .max()
      .rename('NDVI')
      .updateMask(forestMask)
      .clip(roi);

    return annualMax
      .set('year', year)
      .set('system:index', year.format('%d'))
      .set('system:time_start', start.millis());
  })
);

print('年度最大 NDVI 集合', annualNdvi);
print('年度数量', annualNdvi.size());

var ndviStart = annualNdvi
  .filter(ee.Filter.eq('year', START_YEAR))
  .first();
var ndviEnd = annualNdvi
  .filter(ee.Filter.eq('year', END_YEAR))
  .first();

var ndviVis = {
  min: 0.2,
  max: 0.9,
  palette: ['8C510A', 'D8B365', 'F6E8C3', '5AB4AC', '01665E']
};
Map.addLayer(ndviStart, ndviVis, START_YEAR + ' 最大 NDVI', false);
Map.addLayer(ndviEnd, ndviVis, END_YEAR + ' 最大 NDVI');

// -----
// 7. 有效观测年数与线性趋势
// -----
var validYearCount = annualNdvi.select('NDVI').count().rename('valid_years');
var validTrendMask = validYearCount.gte(MIN_VALID_YEARS);

var annualWithTime = annualNdvi.map(function(image) {
  var year = ee.Number(image.get('year'));

```

```

// 从 NDVI 波段派生时间波段，以继承相同投影和有效像元掩膜。
var t = image.select('NDVI')
  .multiply(0)
  .add(year.subtract(START_YEAR))
  .rename('t')
  .float();
return t.addBands(image.select('NDVI').float())
  .copyProperties(image, ['system:time_start', 'year']);
});

// linearFit 的输入顺序必须是 x（时间）在前、y（NDVI）在后。
var linearFit = annualWithTime
  .select(['t', 'NDVI'])
  .reduce(ee.Reducer.linearFit());
var linearSlope = linearFit.select('scale')
  .rename('linear_slope')
  .updateMask(validTrendMask);

// -----
// 8. Sen's slope 与 Kendall tau: 对异常值更稳健的趋势指标
// -----
var senFit = annualWithTime
  .select(['t', 'NDVI'])
  .reduce(ee.Reducer.sensSlope());
var senSlope = senFit.select('slope')
  .rename('sen_slope')
  .updateMask(validTrendMask);

var kendallTau = annualWithTime
  .select(['t', 'NDVI'])
  .reduce(ee.Reducer.kendallsCorrelation(2))
  .rename('kendall_tau')
  .updateMask(validTrendMask);

var slopeVis = {
  min: -0.02,
  max: 0.02,
  palette: ['A50026', 'F46D43', 'FFFFBF', '66BD63', '006837']
};
Map.addLayer(linearSlope, slopeVis, '线性 NDVI 斜率', false);
Map.addLayer(senSlope, slopeVis, 'Sen slope');
Map.addLayer(
  kendallTau,
  {min: -1, max: 1, palette: ['B2182B', 'F7F7F7', '1A9850']},
  'Kendall tau',
  false
);

// -----
// 9. 改善/稳定/退化区域提取
// 分类: -1 退化, 0 稳定, 1 改善
// -----
var improvement = senSlope.gt(SLOPE_THRESHOLD)
  .and(kendallTau.gt(TAU_THRESHOLD));
var degradation = senSlope.lt(-SLOPE_THRESHOLD)
  .and(kendallTau.lt(-TAU_THRESHOLD));

// 用 senSlope*0 创建基底，可保留分析影像的投影和掩膜。
var changeClass = senSlope.multiply(0).toInt8()
  .where(improvement, 1)
  .where(degradation, -1)
  .rename('change_class')
  .updateMask(validTrendMask)
  .clip(roi);

Map.addLayer(
  changeClass,
  {min: -1, max: 1, palette: ['D73027', 'D9D9D9', '1A9850']},
  '森林植被变化分类'
);

```

```

);

// 首尾差值仅用于辅助观察，不替代趋势检验。
var endMinusStart = ndviEnd.subtract(ndviStart)
  .rename('ndvi_change_' + START_YEAR + '_' + END_YEAR)
  .updateMask(validTrendMask);
Map.addLayer(
  endMinusStart,
  {min: -0.3, max: 0.3, palette: ['B2182B', 'F7F7F7', '1A9850']},
  '首尾年度 NDVI 差值',
  false
);

// -----
// 10. 统计四个分区的改善/稳定/退化面积与占比
// -----
var pixelAreaKm2 = ee.Image.pixelArea().divide(1e6).rename('area_km2');

function sumArea(maskImage, geometry) {
  return pixelAreaKm2
    .updateMask(maskImage)
    .reduceRegion({
      reducer: ee.Reducer.sum(),
      geometry: geometry,
      scale: STAT_SCALE,
      maxPixels: 1e9,
      tileScale: 4
    })
    .getNumber('area_km2');
}

var changeStats = zones.map(function(zone) {
  var geometry = zone.geometry();
  var totalArea = sumArea(validTrendMask, geometry);
  var improveArea = sumArea(changeClass.eq(1), geometry);
  var stableArea = sumArea(changeClass.eq(0), geometry);
  var degradeArea = sumArea(changeClass.eq(-1), geometry);

  return zone.set({
    valid_forest_km2: totalArea,
    improvement_km2: improveArea,
    stable_km2: stableArea,
    degradation_km2: degradeArea,
    improvement_pct: improveArea.divide(totalArea).multiply(100),
    stable_pct: stableArea.divide(totalArea).multiply(100),
    degradation_pct: degradeArea.divide(totalArea).multiply(100)
  });
});
print('各分区变化面积与占比', changeStats);

// -----
// 11. 统计各分区逐年的高植被覆盖占比 (NDVI >= 0.60)
// -----
var vegetationStats = ee.FeatureCollection(
  years.map(function(year) {
    year = ee.Number(year);
    var annualImage = annualNdvi
      .filter(ee.Filter.eq('year', year))
      .first();

    return zones.map(function(zone) {
      var geometry = zone.geometry();
      var validMask = annualImage.mask().reduce(ee.Reducer.min());
      var validArea = sumArea(validMask, geometry);
      var vegetationArea = sumArea(
        annualImage.select('NDVI').gte(VEGETATION_NDVI),
        geometry
      );
    });
  });
);

```

```

    return ee.Feature(null, {
      year: year,
      zone_id: zone.get('zone_id'),
      zone_name: zone.get('zone_name'),
      valid_forest_km2: validArea,
      high_vegetation_km2: vegetationArea,
      vegetation_ratio_pct: vegetationArea.divide(validArea).multiply(100)
    });
  });
})
).flatten();
print('各分区逐年高植被覆盖占比', vegetationStats);

// -----
// 12. 时序图与分区占比图
// -----
var ndviSeriesChart = ui.Chart.image.seriesByRegion({
  imageCollection: annualNdvi,
  band: 'NDVI',
  regions: zones,
  reducer: ee.Reducer.mean(),
  scale: STAT_SCALE,
  seriesProperty: 'zone_name',
  xProperty: 'system:time_start'
}).setOptions({
  title: '2018—2026 各分区森林年度最大 NDVI',
  hAxis: {title: '年份', format: 'yyyy'},
  vAxis: {title: '年度最大 NDVI', viewWindow: {min: 0.3, max: 1}},
  lineWidth: 2,
  pointSize: 5
});
print(ndviSeriesChart);

var vegetationRatioChart = ui.Chart.feature.groups({
  features: vegetationStats,
  xProperty: 'year',
  yProperty: 'vegetation_ratio_pct',
  seriesProperty: 'zone_name'
}).setChartType('LineChart').setOptions({
  title: '各分区高植被覆盖占比 (NDVI >= 0.60)',
  hAxis: {title: '年份', format: '####'},
  vAxis: {title: '占比 (%)', viewWindow: {min: 0, max: 100}},
  lineWidth: 2,
  pointSize: 5
});
print(vegetationRatioChart);

// -----
// 13. 添加变化分类图例
// -----
var legend = ui.Panel({
  style: {
    position: 'bottom-left',
    padding: '8px 12px',
    backgroundColor: 'white'
  }
});
legend.add(ui.Label({
  value: '森林植被变化分类',
  style: {fontWeight: 'bold', fontSize: '14px', margin: '0 0 6px 0'}
}));

function addLegendRow(color, label) {
  var colorBox = ui.Label("", {
    backgroundColor: color,
    padding: '8px',
    margin: '0 6px 4px 0'
  });
  var text = ui.Label(label, {margin: '0 0 4px 0'});

```

```

    legend.add(ui.Panel([colorBox, text], ui.Panel.Layout.Flow('horizontal')));
}

addLegendRow('#1A9850', '改善: Sen slope > 0.005 且 tau > 0.40');
addLegendRow('#D9D9D9', '稳定: 未达到改善或退化双阈值');
addLegendRow('#D73027', '退化: Sen slope < -0.005 且 tau < -0.40');
Map.add(legend);

// -----
// 14. 导出: 年度 NDVI、多指标趋势影像、分类、矢量与 CSV 表格
// -----
var annualNdviStack = annualNdvi.toBands().toFloat();
var trendStack = linearSlope
    .addBands(senSlope)
    .addBands(kendallTau)
    .addBands(endMinusStart)
    .addBands(validYearCount.toFloat());

Export.image.toDrive({
    image: annualNdviStack,
    description: 'Annual_Max_NDVI_2018_2026_' + DATA_SOURCE,
    folder: EXPORT_FOLDER,
    fileNamePrefix: 'annual_max_ndvi_2018_2026_' + DATA_SOURCE.toLowerCase(),
    region: roi,
    scale: NATIVE_SCALE,
    maxPixels: 1e13,
    fileFormat: 'GeoTIFF',
    formatOptions: {cloudOptimized: true}
});

Export.image.toDrive({
    image: trendStack.toFloat(),
    description: 'Forest_NDVI_Trend_2018_2026_' + DATA_SOURCE,
    folder: EXPORT_FOLDER,
    fileNamePrefix: 'forest_ndvi_trend_2018_2026_' + DATA_SOURCE.toLowerCase(),
    region: roi,
    scale: NATIVE_SCALE,
    maxPixels: 1e13,
    fileFormat: 'GeoTIFF',
    formatOptions: {cloudOptimized: true}
});

Export.image.toDrive({
    image: changeClass.toInt8(),
    description: 'Forest_Change_Class_2018_2026_' + DATA_SOURCE,
    folder: EXPORT_FOLDER,
    fileNamePrefix: 'forest_change_class_2018_2026_' + DATA_SOURCE.toLowerCase(),
    region: roi,
    scale: NATIVE_SCALE,
    maxPixels: 1e13,
    fileFormat: 'GeoTIFF',
    formatOptions: {cloudOptimized: true}
});

Export.table.toDrive({
    collection: changeStats,
    description: 'Zone_Change_Statistics_2018_2026_' + DATA_SOURCE,
    folder: EXPORT_FOLDER,
    fileNamePrefix: 'zone_change_statistics_2018_2026_' + DATA_SOURCE.toLowerCase(),
    fileFormat: 'CSV',
    selectors: [
        'zone_id', 'zone_name', 'valid_forest_km2',
        'improvement_km2', 'stable_km2', 'degradation_km2',
        'improvement_pct', 'stable_pct', 'degradation_pct'
    ]
});

Export.table.toDrive({
    collection: vegetationStats,

```

```

description: 'Annual_Vegetation_Ratio_2018_2026_' + DATA_SOURCE,
folder: EXPORT_FOLDER,
fileNamePrefix: 'annual_vegetation_ratio_2018_2026_' + DATA_SOURCE.toLowerCase(),
fileFormat: 'CSV',
selectors: [
  'year', 'zone_id', 'zone_name', 'valid_forest_km2',
  'high_vegetation_km2', 'vegetation_ratio_pct'
]
});

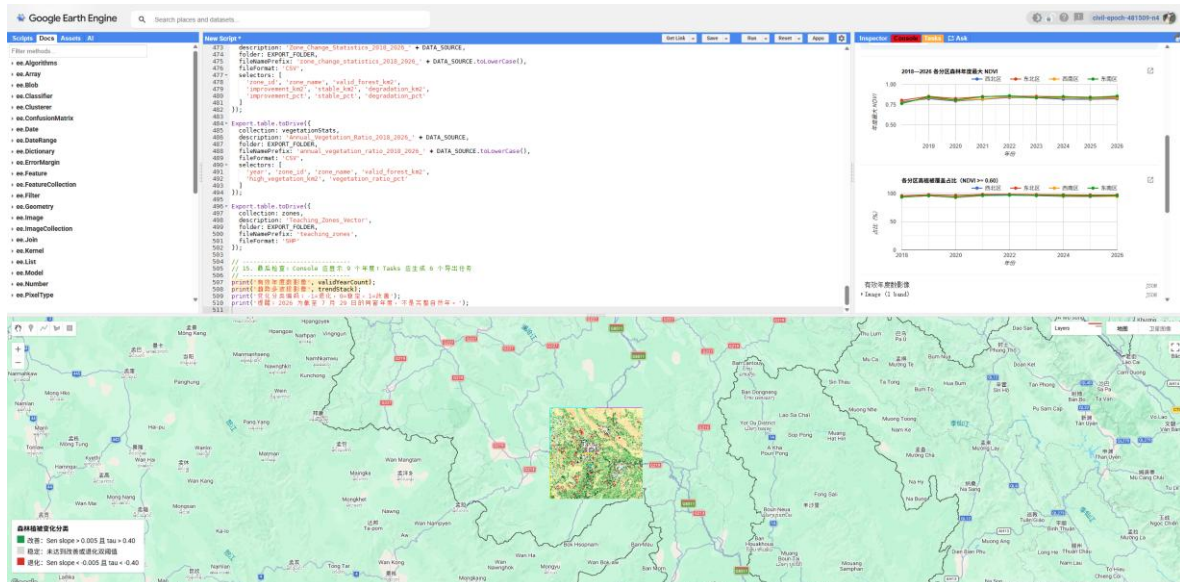
Export.table.toDrive({
  collection: zones,
  description: 'Teaching_Zones_Vector',
  folder: EXPORT_FOLDER,
  fileNamePrefix: 'teaching_zones',
  fileFormat: 'SHP'
});

// -----
// 15. 最后检查：Console 应显示 9 个年度；Tasks 应生成 6 个导出任务
// -----
print('有效年度数影像', validYearCount);
print('趋势多波段影像', trendStack);
print('变化分类编码：-1=退化，0=稳定，1=改善');
print('提醒：2026 为截至 7 月 29 日的同窗年度，不是完整自然年。');

```

4.7.2 运行步骤

1. 新建 Code Editor 脚本，粘贴完整代码。
2. 保持 `DATA_SOURCE='S2'`，点击 Run。
3. Console 确认数据源、影像总数和年度数量 9。
4. 地图打开 2018/2026 NDVI、Sen slope、变化分类。
5. Console 展开 `changeStats` 和 `vegetationStats`，查看是否有 null。
6. 查看两张图表是否正常。
7. Tasks 应出现 6 个导出任务；先提交 CSV，再视存储和配额提交影像。
8. 将 `DATA_SOURCE` 改为 `LANDSAT` 再运行，作为数据源敏感性实验。



4.7.3 参数替换指南

替换 ROI：修改 roi 和 zones；真实分区可从个人 FeatureCollection 资产加载。

更换年份：保证数据源覆盖，并同步比较窗口。

完整 2026：年末把 WINDOW_END_MONTH=12、WINDOW_END_DAY=31，所有年份重新合成。

调整森林定义：替换 forestMask，记录产品年份、类别和阈值。

调整尺度：地图分析可保持原生；区域统计尺度应兼顾精度和算力。

附录 官方资料与数据引用

1. Google Earth Engine. [Earth Engine access](#). 访问核对：2026-07-29。
2. Google Earth Engine. [Noncommercial tiers](#). 访问核对：2026-07-29。
3. Google Earth Engine. [Earth Engine Code Editor](#). 访问核对：2026-07-29。
4. Google Earth Engine Data Catalog. [Sentinel-2 SR Harmonized](#).
5. Google Earth Engine Data Catalog. [Landsat 8 Collection 2 Tier 1 Level 2](#).
6. Google Earth Engine Data Catalog. [ESA WorldCover v200](#).
7. Google Earth Engine API. [ui.Chart.image.series](#).
8. Google Earth Engine API. [Export.image.toDrive](#); [Export.table.toDrive](#).
9. Google Earth Engine API. [ee.Reducer.linearFit](#); [ee.Reducer.sensSlope](#); [ee.Reducer.kendallsCorrelation](#).
10. Google Earth Engine Community Tutorial. [Non-Parametric Trend Analysis](#). 该教程为社区内容，课程仅将其作为趋势方法延伸阅读。
11. Google Earth Engine. [Managing assets](#); [Table upload](#); [Image upload](#). 访问核对：2026-07-30。
12. Google Earth Engine. [Access control](#); [Earth Engine Apps](#). 访问核对：2026-07-30。
13. Google Earth Engine API. [Map.addLayer](#); [FeatureCollection.style](#); [Image visualization](#).
14. Google Earth Engine API. [Export.image.toAsset](#); [Exporting images](#).
15. OpenAI. [Model guidance—Prompting best practices](#). 用于任务目标、上下文、约束、证据、成功标准和输出格式的提示原则。